

ДИПЛОМНА РАБОТА



ПРОЕКТИРАНЕ И РАЗРАБОТВАНЕ НА БИБЛИОТЕКА НА C++, ПОДПОМАГАЩА СЪЗДАВАНЕТО НА УЕБ ПРИЛОЖЕНИЯ

Coroute: Coroutine-Based Multi-Protocol Web Framework

Изготвил	Факултетен номер	Специалност
Алекс Цветанов	121222225	Компютърно и софтуерно инженерство

Научен ръководител
доц. д-р инж. Иван Станков
Катедра "Киберсигурност"

Технически университет – София, 5 март 2026 г.

Съдържание

1	Въведение	8
1.1	Актуалност на проблема	8
1.2	Цели и задачи	8
1.3	Обхват и ограничения	10
1.4	Структура на дипломната работа	11
2	Преглед на съществуващи решения	13
2.1	Boost.Beast	13
2.2	Drogon	13
2.3	Crow	14
2.4	Oat++	14
2.5	Pistache	15
2.6	Сравнение с C++ UI Фреймуърци (Qt, Slint)	15
2.7	Сравнителен анализ	16
2.8	Мотивация за Coroute	16
3	Теоретична основа	18
3.1	HTTP протокол	18
3.1.1	HTTP/1.1	18
3.1.2	HTTP/2	19
3.2	WebSocket протокол	20
3.3	TLS и сигурност	21
3.4	C++20 корутини	23
3.4.1	Основни концепции	23
3.4.2	Предимства на корутините	24
3.5	Асинхронен I/O	26
3.5.1	IOCP (Windows)	26
3.5.2	io_uring (Linux)	27
3.5.3	kqueue (macOS/BSD)	27
3.6	DFA-базирано маршрутизиране	28
3.6.1	Теоретична основа	29
3.6.2	Алгоритъм за изграждане на pattern graph	29
3.6.3	Приложение в Coroute	30
3.6.4	Benchmark резултати	30
3.7	Foreign Function Interface (FFI)	30
3.7.1	Паметови граници и управление	31
3.7.2	Контекст на изпълнение и асинхронност	31
3.8	Шаблонни машини (Template Engines)	31
4	Архитектура на Coroute	32
4.1	Обща архитектура и модулна структура	32
4.2	Жизнен цикъл на HTTP заявка	33
4.3	Корутинен модел на изпълнение – Task<T>	35
4.3.1	Promise Type	35
4.3.2	FinalAwaiter	35
4.3.3	Awaiter	36
4.4	Безопасност на паметта при detached корутини	36

4.4.1	Проблемът	36
4.4.2	Решения в Coroute	37
4.4.3	Правила за безопасност	38
4.5	DFA-базирано маршрутизиране	38
4.5.1	Регистрация на маршрути	38
4.5.2	Конвертиране на шаблони	38
4.5.3	Съпоставяне с $O(N)$ сложност	39
4.6	Middleware верига	39
4.6.1	Дефиниция на Middleware	39
4.6.2	Рекурсивно изпълнение	39
4.6.3	Пример за middleware	40
4.7	Платформено-независим I/O слой	40
4.7.1	IoContext интерфейс	40
4.7.2	Connection интерфейс	40
4.7.3	Платформени имплементации	41
4.8	Graceful Shutdown	41
4.9	Headless Web Engine и FFI Модел на Изпълнение	41
4.9.1	C API Изолация	42
4.9.2	Вътрешни заявки (Internal Sub-requests)	42
5	Реализация на протоколи	43
5.1	HTTP/1.1 парсер и сериализация	43
5.1.1	Парсване на заявки	43
5.1.2	URL декодиране	44
5.1.3	Keep-Alive поддръжка	45
5.1.4	Zero-copy file transfer	45
5.2	HTTP/2 поддръжка	46
5.2.1	ALPN	46
5.2.2	HTTP/2 Connection	47
5.2.3	h2c Upgrade	47
5.3	WebSocket протокол	48
5.3.1	Проверка за WebSocket upgrade	48
5.3.2	Изчисляване на Accept Key	48
5.3.3	Upgrade процес	49
5.3.4	WebSocket handler регистрация	49
5.3.5	WebSocket съобщения	50
5.4	TLS интеграция	50
5.4.1	TLS конфигурация	50
5.4.2	TLS Listener	51
6	Типово-безопасно извличане на параметри	52
6.1	Мотивация	52
6.2	Template-базирани route handlers	53
6.2.1	Регистрация с типови параметри	53
6.2.2	Генериране на wrapper handler	53
6.2.3	Извличане и извикване	53
6.3	FromString<T> trait система	54
6.3.1	Основна структура	54
6.3.2	Специализация за целочислени типове	54

6.3.3	Специализация за floating-point типове	55
6.3.4	Специализация за std::string	55
6.3.5	Потребителски типове	55
6.4	Compile-time проверка на типове	56
6.5	Runtime валидация	57
6.6	Множество параметри	57
6.7	Анализ на предимствата	57
6.7.1	Коректност и надеждност	57
6.7.2	Производителност	57
6.7.3	Разширяемост	58
6.7.4	Документация чрез типове	58
6.8	Сравнение с други библиотеки	58
6.9	Заклучение	58
7	Изгледи и Шаблонизиране (View & Templating)	59
7.1	Интеграция с Inja	59
7.2	Компоненти на View Системата	59
7.2.1	ViewTemplates кеширане	59
7.2.2	ViewRouteInfo за конфигурация	59
7.2.3	Лесно конструиране чрез ResponseBuilder	59
7.3	Изпълнение на шаблонизирани маршрути	60
8	Flutter Интеграция и Headless Изпълнение	61
8.1	Взаимодействие между C++ и Dart (FFI)	61
8.2	Инструментариум за интеграция (Tooling)	61
8.2.1	coroute_framework	62
8.2.2	CorouteApp.cmake и .flutter пясъчник	62
8.3	Управление на събития през езиковата граница	62
9	Примерно приложение	63
9.1	Описание на приложението	63
9.2	Структура на проекта (Flutter Enterprise App)	63
9.3	Конфигурация	64
9.3.1	Config клас	64
9.3.2	Server setup	65
9.4	Middleware	65
9.4.1	Logging middleware	65
9.4.2	Authentication middleware	66
9.5	REST API handlers	66
9.5.1	Task CRUD	66
9.6	WebSocket handler	68
9.6.1	TaskHub клас	68
9.6.2	WebSocket route	68
9.7	HTML шаблони	69
9.7.1	Template rendering	69
9.7.2	Примерен шаблон	69
9.8	JavaScript клиент	70
9.9	Entry point	71
9.10	Унификация между Web и Flutter (FFI Entry Point)	71

9.11	Анализ на архитектурните решения	73
9.11.1	Корутинен модел за WebSocket	73
9.11.2	Типово-безопасни route параметри	73
9.11.3	Middleware композиция	73
9.12	Сравнение с други библиотеки	73
9.13	Заклучение	73
10	Тестване и производителност	75
10.1	Методология на експерименталната оценка	75
10.1.1	Хардуерна конфигурация	75
10.1.2	Параметри на тестовете	75
10.1.3	Инструменти за benchmark	75
10.1.4	Ограничения на методологията	75
10.2	Unit тестове	76
10.2.1	Тестване на Router	76
10.2.2	Тестване на FromString	77
10.2.3	Тестване на Task<T>	77
10.3	Integration тестове	78
10.4	Тестване на FFI Границите	79
10.5	Benchmark резултати	79
10.5.1	Сценарий 1: Hello World	80
10.5.2	Сценарий 2: JSON API	81
10.5.3	Сценарий 3: Статичен файл	81
10.5.4	Сценарий 4: Маршрут с параметри	81
10.5.5	Сравнителна таблица	82
10.6	DFA маршрутизиране – производителност	82
10.6.1	Тест с множество маршрути	82
10.7	Анализ на латентност	83
10.7.1	Percentile разпределение	83
10.8	Консумация на памет	84
10.9	Linux io_uring резултати	84
10.9.1	Верификация на io_uring backend	84
10.9.2	Резултати при нормални условия	85
10.9.3	Сравнение Windows IOCP vs Linux io_uring	85
10.10	Мрежова устойчивост	85
10.10.1	Методология	85
10.10.2	Резултати при деградирана мрежа	85
10.10.3	Анализ на резултатите	86
10.10.4	Консумация на памет при стрес тест	86
10.11	Windows мрежова симулация	86
10.11.1	Методология	86
10.11.2	Резултати при деградирана мрежа (Windows)	87
10.12	Сравнителен анализ: платформи и мрежови условия	87
10.12.1	Сравнение Windows vs Linux при нормални условия	87
10.12.2	Сравнение localhost vs деградирана мрежа	88
10.13	Статистически анализ на резултатите	88
10.14	Заклучения от експерименталната оценка	88
11	Заклучение	90

11.1	Обобщение на постигнатите резултати	90
11.1.1	Архитектурни постижения	90
11.1.2	Експериментални резултати	91
11.1.3	Практическа приложимост	91
11.2	Научен принос	91
11.3	Ограничения на изследването	92
11.4	Насоки за бъдещо развитие	92
11.4.1	HTTP/3 и QUIC	92
11.4.2	Compile-Time Query Builder	92
11.4.3	Допълнителни направления	93
11.5	Заключителни думи	93

Литература	94
-------------------	-----------

Резюме

Настоящата дипломна работа представя проектирането, реализацията и експерименталната оценка на Coroute — високопроизводителна C++ библиотека за изграждане на уеб приложения, базирана на съвременните възможности на стандарта C++20. Основната цел на изследването е да се демонстрира, че комбинацията от stackless корутини, операционно-специфични асинхронни I/O механизми и алгоритмично оптимизирано маршрутизиране може да постигне производителност, съизмерима с водещите индустриални решения, при значително опростен програмен модел.

Архитектурата на Coroute се основава на няколко ключови иновации. Първо, изпълнителният модел използва типа `Task<T>`, който реализира lazy корутини с поддръжка на detached изпълнение и кооперативно прекратяване. Този модел елиминира традиционните недостатъци на callback-базираното асинхронно програмиране, като запазва ефективността на неблокиращия I/O. Второ, мрежовият слой абстрахира платформено-специфичните механизми — IOCP за Windows и `io_uring` за Linux — зад унифициран интерфейс, позволявайки zero-copy операции и batching на системни извиквания. Трето, маршрутизирането използва DFA-базиран алгоритъм, публикуван в „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1], който постига $O(N)$ времева сложност спрямо дължината на URL-а, независимо от броя на регистрираните маршрути. Четвърто, архитектурата включва шаблонна система за изгледи (Template Views) и безпроблемна интеграция с Flutter чрез Dart FFI, което позволява преизползването на библиотеката като headless web engine за изграждане на нейтив мобилни и десктоп приложения.

Библиотеката поддържа множество протоколи върху един порт чрез ALPN негоциация: HTTP/1.1 с keep-alive, HTTP/2 с HPACK компресия и stream мултиплексиране, и WebSocket за двупосочна комуникация в реално време. Типово-безопасното извличане на параметри използва C++20 concepts за compile-time валидация, елиминирайки цял клас runtime грешки. Шаблонната система за изгледи разширява възможностите за сървърно рендиране чрез интеграция с шаблонизатора Inja.

Експерименталната оценка, проведена на система с AMD Ryzen 5 3600 (6 ядра, 12 нишки) и до 1024 конкурентни клиента, демонстрира throughput от 1,378,578 заявки в секунда за прости HTTP отговори, с медианна латентност от 212 микросекунди и 0% error rate. Сравнителният анализ с Crow, Boost.Beast и Drogon показва, че Coroute постига сравнима или по-добра производителност при значително по-прост API.

Научният принос на работата включва: интеграция на DFA-базирано маршрутизиране в пълноценен уеб библиотека; корутинен изпълнителен модел, специално проектиран за мрежови приложения; и демонстрация на практическата приложимост на подхода чрез production-scale примерно приложение.

Ключови думи: C++20, корутини, уеб сървър, HTTP/2, WebSocket, IOCP, io_uring, DFA маршрутизиране, типова безопасност, асинхронен I/O, Flutter, Dart FFI, Template Views, Inja

1 Въведение

1.1 Актуалност на проблема

През последните две десетилетия уеб технологиите претърпяха драматична трансформация. От статични HTML страници, обслужвани от прости CGI скриптове, се премина към сложни интерактивни приложения, работещи в реално време. Съвременните уеб услуги трябва да обработват хиляди, а понякога и милиони едновременни връзки, като същевременно поддържат латентност от порядъка на милисекунди.

Традиционните подходи за изграждане на уеб сървъри се базират на модела “една нишка на връзка” (thread-per-connection). При този модел всяка входяща връзка се обслужва от отделна нишка, която блокира докато чака данни от клиента или от базата данни. Макар и концептуално прост, този подход се сблъсква със сериозни ограничения при мащабиране. Създаването на нишка консумира значителни системни ресурси – типично около 1MB памет за стека. При хиляди едновременни връзки консумацията на памет става неприемлива. Освен това, контекстното превключване между нишките въвежда допълнителен overhead, който намалява общата производителност на системата.

Тези ограничения мотивираха разработването на асинхронни модели на изпълнение. При асинхронния подход малък брой нишки (обикновено равен на броя на CPU ядрата) обслужва множество връзки, като използва неблокиращи I/O операции и event loops. Когато една операция не може да завърши веднага, вместо да блокира нишката, тя регистрира callback функция, която ще бъде извикана при завършване на операцията. Междувременно нишката може да обслужва други заявки.

Езикът C++ остава предпочитан избор за системно програмиране и приложения с високи изисквания към производителността. Неговата способност да генерира оптимизиран машинен код, съчетана с богатата стандартна библиотека и възможности за низкониво програмиране, го правят идеален за изграждане на високопроизводителни мрежови приложения.

C въвеждането на корутините в стандарта C++20 [2] се открива нова възможност за създаване на асинхронен код, който е едновременно ефективен и четим. Корутините са функции, които могат да спрат изпълнението си в определени точки и да го възобновят по-късно, без да блокират нишката. Това позволява писане на код, който изглежда синхронен и последователен, но всъщност се изпълнява асинхронно. По този начин се елиминира т.нар. “callback hell” – проблемът с дълбоко вложени callback функции, характерен за традиционните асинхронни API-та.

1.2 Цели и задачи

Основната цел на настоящата дипломна работа е проектирането и реализацията на модерна C++ библиотека за създаване на уеб приложения, която да отговаря на съвременните изисквания за производителност, надеждност и удобство при разработка.

Библиотеката трябва да **опростява разработката** на уеб приложения чрез интуитивен API, базиран на C++20 корутини. Разработчиците трябва да могат да пишат асинхронен код със синтаксис, подобен на синхронния, без да се налага да управляват ръчно callbacks или promises.

Библиотеката трябва да **осигурява висока производителност** чрез неблокиращ I/O и ефективно управление на ресурсите. Целта е да се постигне производителност, сравнима с водещите C++ уеб библиотеки, като същевременно се запази простотата на API-то.

Библиотеката трябва да **поддържа множество протоколи**, включително HTTP/1.1, HTTP/2 и WebSocket. HTTP/2 е особено важен за съвременните уеб приложения, тъй като позволява мултиплексиране на заявки и ефективна компресия на хедъри. WebSocket е необходим за приложения, изискващи двупосочна комуникация в реално време.

Библиотеката трябва да **предоставя типова безопасност** при извличане на параметри от URL адреси. Вместо да връща параметрите като низове, които трябва да се конвертират ръчно, библиотеката трябва автоматично да извлича стойности от правилния тип, като грешките се откриват по време на компилация.

Библиотеката трябва да бъде **платформено независима**, работеща на Windows, Linux и macOS. За всяка платформа трябва да се използват оптимизиранияте native механизми за асинхронен I/O.

Библиотеката трябва да бъде **разширяема**, позволявайки лесно добавяне на нови протоколи и middleware компоненти. Архитектурата трябва да следва принципите на модулност и разделение на отговорностите.

За постигане на тези цели са поставени следните конкретни задачи:

Първата задача е **анализ на съществуващи C++ уеб библиотеки**. Необходимо е да се проучат популярните решения като Boost.Beast, Drogon, Crow и други, да се идентифицират техните предимства и недостатъци, и да се определи какви функционалности липсват на пазара.

Втората задача е **проектиране на модулна архитектура** с ясно разделение на отговорностите. Архитектурата трябва да позволява независимо развитие на отделните компоненти и лесно добавяне на нови функционалности.

Третата задача е **реализация на корутинна инфраструктура**. Необходимо е да се създаде типът `Task<T>`, който да служи като основа за асинхронните операции в библиотеката.

Четвъртата задача е **имплементация на DFA-базиран маршрутизатор** с $O(N)$ сложност, базиран на алгоритъма, описан в „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1]. Този алгоритъм позволява ефективно съпоставяне на URL адреси с множество шаблони.

Петата задача е **разработка на платформено-специфични I/O бекенди** за IOCP (Windows), io_uring (Linux) и kqueue (macOS). Всеки бекенд трябва да използва оптимално native механизмите на съответната операционна система.

Шестата задача е **интеграция на HTTP/2** с ALPN негоциация и HPACK компресия на хедъри. HTTP/2 трябва да работи безпроблемно заедно с HTTP/1.1, като протоколът се избира автоматично при TLS handshake.

Седмата задача е **реализация на WebSocket протокол** за двупосочна комуникация. WebSocket трябва да поддържа както текстови, така и бинарни съобщения.

Осмата задача е **тестване и benchmark анализ** на производителността. Необходимо е да се създадат unit тестове за всички компоненти и да се измери производителността в сравнение с други библиотеки.

1.3 Обхват и ограничения

При проектирането на всяка софтуерна система е важно ясно да се дефинира какво влиза в обхвата на разработката и какво остава извън него. Това позволява фокусиране на усилията върху ключовите функционалности и избягване на т.нар. “scope creep” – неконтролираното разширяване на изискванията.

Библиотеката Coroute е проектирана като *library*, а не като самостоятелен сървър или framework. Това е съзнателно архитектурно решение, което дава на разработчиците максимална гъвкавост. Библиотеката предоставя градивни блокове за създаване на уеб приложения, но оставя контрола върху конфигурацията, разгръщането и интеграцията с други системи изцяло на разработчика.

В обхвата на настоящата разработка влизат следните функционалности:

Поддръжката на **HTTP/1.1 и HTTP/2 протоколи** е основна функционалност на библиотеката. HTTP/1.1 е все още широко използван и трябва да се поддържа за съвместимост. HTTP/2 предоставя значителни подобрения в производителността и е необходим за съвременни приложения.

WebSocket протоколът позволява двупосочна комуникация между клиент и сървър. Това е необходимо за приложения като чат системи, игри в реално време, колаборативни редактори и dashboard-и с live updates.

TLS криптирането е задължително за съвременните уеб приложения. Библиотеката трябва да поддържа TLS 1.2 и TLS 1.3, включително ALPN негоциация за избор на HTTP протокол.

Статичното обслужване на файлове е често срещано изискване. Библиотеката трябва да поддържа ефективно обслужване на статични ресурси като CSS, JavaScript и изображения, включително zero-copy трансфер където е възможно.

Middleware системата позволява добавяне на cross-cutting concerns като

логване, автентикация, компресия и rate limiting. Middleware компонентите трябва да могат да се композират в произволен ред.

Маршрутизирането с параметри позволява дефиниране на динамични URL шаблони като `/users/{id}`. Параметрите трябва да се извличат автоматично и да се валидират по тип.

Шаблонна система за генериране на HTML (Template Views) е интегрирана в архитектурата, използвайки Inja за ефективно рендиране на динамични уеб страници директно от сървъра.

Интеграцията с Flutter чрез Dart FFI предоставя възможност C++ ядрото да функционира като headless web engine, позволявайки създаването на крос-платформени (мобилни и десктоп) приложения с единна логика и локално изпълнение, без нужда от отдалечен мрежов сървър.

Извън обхвата на настоящата разработка остават следните функционалности:

HTTP/3 (QUIC) е най-новата версия на HTTP протокола, базирана на UDP вместо TCP. Въпреки че HTTP/3 предоставя допълнителни подобрения в производителността, неговата имплементация е значително по-сложна и е планирана за бъдещи версии на библиотеката.

Вградена база данни или ORM не е част от обхвата. Библиотеката се фокусира върху мрежовия слой и оставя избора на database решение на разработчика. Това позволява интеграция с произволна база данни – SQL, NoSQL или in-memory.

Автоматичното мащабиране и load balancing са функционалности на инфраструктурно ниво, които обикновено се реализират чрез външни инструменти като Kubernetes, nginx или Nginx Proxy.

1.4 Структура на дипломната работа

Дипломната работа е организирана в следните глави:

Глава 2 представя преглед на съществуващите C++ уеб библиотеки, техните характеристики и сравнителен анализ.

Глава 3 въвежда теоретичната основа – HTTP протоколи, WebSocket, TLS, C++20 корутини, асинхронен I/O и DFA-базирано маршрутизиране.

Глава 4 описва архитектурата на Coroute – модулна структура, жизнен цикъл на заявките, корутинен модел и middleware верига.

Глава 5 разглежда реализацията на поддържаните протоколи – HTTP/1.1, HTTP/2 и WebSocket.

Глава 6 представя системата за типово-безопасно извличане на параметри от

URL адреси.

Глава 7 описва шаблонната система за изгледи и интеграцията с Inja.

Глава 8 разглежда интеграцията с Flutter чрез Dart FFI и архитектурата на headless web engine.

Глава 9 демонстрира примерно приложение, изградено с библиотеката, едновременно като самостоятелен сървър и като Flutter клиент.

Глава 10 съдържа резултатите от тестването, включително тестване на FFI границите, и анализ на производителността.

Заклучението обобщава постигнатите резултати и очертава насоки за бъдещо развитие.

2 Преглед на съществуващи решения

Анализът на съществуващите решения е критична стъпка в процеса на проектиране на нова софтуерна система. Той позволява да се идентифицират утвърдени архитектурни модели, да се избегнат известни проблеми и да се определят области, в които новото решение може да предостави добавена стойност. В контекста на C++ уеб библиотеки, този анализ е особено важен поради разнообразието от подходи — от низкониво мрежово програмиране до високониво абстракции, подобни на тези в интерпретираните езици.

Настоящата глава представя систематичен преглед на пет водещи C++ библиотеки за уеб разработка: Boost.Beast, Drogon, Crow, Oat++ и Pistache. За всяка библиотека се анализират архитектурните решения, моделът на конкурентност, API дизайнът и документираната производителност. Анализът завършва със сравнителна таблица и обосновка на мотивацията за разработване на Coroute.

2.1 Boost.Beast

Boost.Beast [3] е част от утвърдената колекция Boost библиотеки и представлява една от най-зрелите имплементации на HTTP и WebSocket протоколите за C++. Библиотеката е изградена върху Boost.Asio, което ѝ осигурява солидна основа за асинхронни мрежови операции.

Философията на Boost.Beast е да предостави ниско ниво на абстракция, давайки на разработчика пълен контрол върху всеки аспект на HTTP комуникацията. Това означава, че библиотеката не налага конкретна архитектура или модел на използване, а вместо това предоставя градивни блокове, от които разработчикът може да изгради своето решение.

Едно от основните предимства на Boost.Beast е нейната интеграция с Boost екосистемата. Разработчиците, които вече използват други Boost библиотеки, могат лесно да добавят HTTP функционалност към своите проекти. Документацията е изчерпателна и включва множество примери за различни сценарии на използване. Активната общност осигурява бърза помощ при възникнали проблеми.

Въпреки тези предимства, Boost.Beast има и съществени недостатъци. Ниското ниво на абстракция означава, че дори прости задачи изискват значително количество код. Липсата на вградено маршрутизиране принуждава разработчиците да имплементират собствена логика за насочване на заявките. Няма middleware система, което затруднява добавянето на cross-cutting concerns като логване, автентикация или компресия. Кривата на обучение е стръмна, особено за разработчици без опит с Boost.Asio.

2.2 Drogon

Drogon [4] се е утвърдил като един от най-бързите C++ уеб библиотеки, редовно заемащ водещи позиции в TechEmpower Web Framework Benchmarks [5]. Библиотеката е разработен с фокус върху производителността и предоставя пълен набор от функционалности за изграждане на съвременни уеб приложения.

Архитектурата на Drogon е базирана на неблокиращ I/O модел, използващ

event loop за обработка на множество едновременни връзки. Библиотеката поддържа HTTP/1.1, HTTP/2 и WebSocket протоколи, което го прави подходящ за широк спектър от приложения. Вградената ORM система улеснява работата с бази данни, като поддържа PostgreSQL, MySQL и SQLite.

Една от силните страни на Drogon е неговата middleware система, която позволява добавяне на филтри за обработка на заявките. Библиотеката предоставя и инструменти за автоматично генериране на контролери от дефиниции, което ускорява разработката. Документацията е добра, макар и предимно на английски и китайски език.

Въпреки впечатляващата производителност, Drogon има някои недостатъци. Асинхронният модел е базиран на callbacks, което може да доведе до т.нар. “callback hell” при сложна логика. Въпреки че последните версии добавят частична поддръжка на C++20 корутини, тя не е пълноценна и не покрива всички аспекти на библиотеката. Конфигурацията може да бъде сложна за начинаещи, а размерът на библиотеката е значителен, което увеличава времето за компилация.

2.3 Crow

Crow [6] е микробиблиотека, вдъхновен от популярния Python библиотека Flask. Основната цел на Crow е да предостави минималистичен и интуитивен API, който позволява бързо създаване на уеб приложения с минимално количество код.

Една от най-привлекателните характеристики на Crow е неговата простота. Библиотеката е реализиран като header-only библиотека, което означава, че интеграцията в проект се свежда до включване на един header файл. Маршрутизирането се извършва чрез lambda функции, което прави кода четим и лесен за разбиране. За разработчици, идващи от Python или JavaScript екосистемите, синтаксисът на Crow ще изглежда познат и интуитивен.

Crow е особено подходящ за малки проекти, прототипи и вътрешни инструменти, където простотата е по-важна от производителността. Библиотеката поддържа WebSocket комуникация и предоставя базова middleware функционалност.

Въпреки своята елегантност, Crow има съществени ограничения. Липсата на HTTP/2 поддръжка го прави неподходящ за съвременни production приложения, които изискват мултиплексиране и ефективна компресия на хедъри. Синхронният модел по подразбиране означава, че при високо натоварване производителността страда значително. Библиотеката не предоставя вградени инструменти за работа с бази данни, сесии или автентикация, което налага използването на допълнителни библиотеки.

2.4 Oat++

Oat++ [7] представлява модерен подход към C++ уеб разработката, поставяйки силен акцент върху типовата безопасност и автоматизацията. Библиотеката е проектиран с идеята, че API документацията трябва да се генерира автоматично от кода, а не да се поддържа ръчно.

Централна концепция в Oat++ е системата от Data Transfer Objects (DTO). Разработчиците дефинират структурите на данните чрез специални макроси, които позволяват автоматична сериализация и десериализация към JSON, както и генериране на OpenAPI/Swagger документация. Това елиминира честите грешки при ръчно писане на документация и гарантира, че тя винаги е синхронизирана с кода.

Библиотеката предоставя както синхронен, така и асинхронен API, давайки на разработчиците гъвкавост при избора на модел на изпълнение. Вградената система за валидация на входни данни допълнително подобрява надеждността на приложенията.

Основният недостатък на Oat++ е неговата собствена система от типове. Вместо да използва стандартни C++ типове, библиотеката изисква използването на специални wrapper класове, което може да бъде объркващо за начинаещи. Кривата на обучение е по-стръмна в сравнение с по-простите библиотеки. Липсата на HTTP/2 поддръжка е съществен пропуск за приложения, изискващи висока производителност при множество едновременни заявки.

2.5 Pistache

Pistache [8] е библиотека, специализиран в създаването на RESTful API услуги. Проектиран е с фокус върху елегантността на кода и лесната употреба, следвайки принципите на модерния C++ дизайн.

API-то на Pistache е чисто и изразително, позволявайки дефиниране на endpoints с минимално количество boilerplate код. Библиотеката използва асинхронен модел на изпълнение, базиран на Linux epoll, което осигурява добра производителност при обработка на множество едновременни връзки. Вградената поддръжка за HTTP методи, статус кодове и content negotiation улеснява изграждането на стандартни REST API-та.

Pistache предоставя и допълнителни функционалности като rate limiting, timeout management и graceful shutdown, които са важни за production deployments. Документацията е ясна и включва примери за често срещани сценарии.

Най-съществените ограничения на Pistache са свързани с платформената поддръжка и функционалността. Библиотеката работи само на Linux, което го прави неподходящ за проекти, изискващи cross-platform съвместимост. Липсата на HTTP/2 и WebSocket поддръжка ограничава приложимостта му в съвременни уеб приложения. Активността на проекта е намаляла през последните години, което поражда въпроси относно дългосрочната поддръжка и развитие.

2.6 Сравнение с C++ UI Фреймуърци (Qt, Slint)

Традиционният подход за изграждане на графични потребителски интерфейси (GUI) на C++ включва използването на специализирани фреймуърци като Qt или по-модерни алтернативи като Slint. Qt предоставя огромна екосистема и собствена мета-обектна система (MOC), но силно обвързва бизнес логиката със своя специфичен парадигмен модел. Това често затруднява преизползването на логика между отдалечени бекенд услуги и локални клиентски приложения. Slint предлага

по-модерен декларативен синтаксис, но все пак изисква специфични инструменти и усвояване на нов език за дизайн.

За разлика от тях, интеграцията на Coroute с Flutter предлага иновативен архитектурен подход. Вместо C++ да управлява потребителския интерфейс, C++ ядрото се компилира като споделена библиотека и функционира като headless web engine локално на устройството. Цялата бекенд логика (маршрутизиране, контролери, бази данни, WebSocket) остава идентична с тази на мрежовия сървър. Flutter поема единствено фронтенд презентацията. Връзката се осъществява чрез високопроизводителен Dart FFI (Foreign Function Interface), трансферирайки състояния и съобщения с минимален overhead. Този подход обединява абсолютната производителност на C++ бекенда с богатата екосистема на Flutter за създаване на 60fps крос-платформени (iOS, Android, Windows, macOS, Linux) приложения.

2.7 Сравнителен анализ

След детайлния преглед на всеки библиотека, е полезно да направим систематично сравнение по ключови характеристики. Таблица 1 обобщава поддръжката на различни функционалности от всеки от разгледаните библиотеки, включително Coroute.

Таблица 1: Сравнение на C++ уеб библиотеки по ключови характеристики

Характеристика	Beast	Drogon	Crow	Oat++	Pistache	Coroute
HTTP/1.1	✓	✓	✓	✓	✓	✓
HTTP/2	–	✓	–	–	–	✓
WebSocket	✓	✓	✓	✓	–	✓
C++20 корутини	–	частично	–	–	–	✓
Middleware	–	✓	✓	✓	✓	✓
Типово маршрут.	–	–	–	✓	–	✓
Cross-platform	✓	✓	✓	✓	–	✓

От таблицата се вижда, че всеки библиотека има своите силни страни, но нито един не предоставя пълния набор от желани функционалности. Boost.Beast е мощен, но ниско ниво. Drogon е бърз, но callback-базиран. Crow е прост, но ограничен. Oat++ е типово-безопасен, но без HTTP/2. Pistache е елегантен, но само за Linux.

2.8 Мотивация за Coroute

Анализът на съществуващите решения разкрива съществена празнина на пазара. Няма C++ библиотека, която да комбинира всички следни характеристики в едно цялостно решение:

Първо, **пълноценна поддръжка на C++20 корутини**. Корутините са революционна функционалност, която позволява писане на асинхронен код със синтаксис, подобен на синхронния. Въпреки че C++20 е наличен от 2020 година, повечето библиотеки все още използват callback-базирани модели или предоставят само частична поддръжка на корутини.

Второ, **HTTP/2 с ALPN негоциация**. HTTP/2 е стандарт от 2015 година и

предоставя значителни подобрения в производителността чрез мултиплексиране на потоци и компресия на хедъри. Въпреки това, много C++ библиотеки все още не го поддържат или изискват сложна конфигурация.

Трето, **типово-безопасно маршрутизиране с compile-time проверки**. Традиционните библиотеки извличат параметри от URL като низове, оставяйки валидацията на runtime. Използването на C++ шаблони позволява проверка на типовете по време на компилация, елиминирайки цял клас грешки.

Четвърто, **DFA-базирано маршрутизиране с $O(N)$ сложност**. Повечето библиотеки използват последователно съпоставяне на маршрути, което води до $O(N \times M)$ сложност. Алгоритъмът, описан в „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1], позволява съпоставяне с линейна сложност спрямо дължината на URL-а, независимо от броя на маршрутите.

Пето, **платформена независимост с оптимизирани I/O бекенди**. Различните операционни системи предоставят различни механизми за асинхронен I/O – IOCP на Windows, io_uring на Linux, kqueue на macOS. Оптималната производителност изисква използване на native механизми за всяка платформа.

Шесто, **интеграция с крос-платформени UI фреймуърци като Flutter**. Чрез възможността за компилиране на уеб логиката като споделена библиотека, Coroute преодолява границата между типичния сървърен софтуер и локалните клиентски приложения чрез FFI комуникация.

Coroute е създаден с амбицията да запълни тази ниша. Библиотеката комбинира всички изброени характеристики в едно цялостно, модерно и ефективно решение за C++ уеб разработка. В следващите глави ще разгледаме детайлно архитектурата и имплементацията на всяка от тези функционалности.

3 Теоретична основа

Преди да пристъпим към описание на архитектурата и имплементацията на Coroute, е необходимо да представим теоретичните концепции, върху които е изградена библиотеката. Тази глава обхваща HTTP протоколите, WebSocket, TLS криптирането, C++20 корутините, асинхронния I/O и DFA-базираното маршрутизиране.

Разбирането на тези концепции е от съществено значение за правилното използване на библиотеката и за вземане на информирани архитектурни решения при изграждане на уеб приложения.

3.1 HTTP протокол

Hypertext Transfer Protocol (HTTP) е основният протокол за комуникация в World Wide Web. От създаването си през 1991 година, HTTP е претърпял значителна еволюция, като всяка нова версия адресира ограниченията на предходната.

3.1.1 HTTP/1.1

HTTP/1.1, дефиниран в RFC 2616 [9], е текстово-базиран протокол, използващ модела заявка-отговор (request-response). При този модел клиентът изпраща заявка към сървъра, а сървърът отговаря с резултата. Всяка заявка е независима и не съхранява състояние между отделните обмени (stateless протокол).

HTTP/1.1 въведе редица подобрения спрямо HTTP/1.0, включително persistent connections (keep-alive), chunked transfer encoding и подобрена кеширане. Въпреки тези подобрения, протоколът има фундаментални ограничения, които стават все по-проблематични при съвременните уеб приложения.

Структура на HTTP заявка:

```
GET /path/to/resource HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

Структура на HTTP отговор:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234
```

```
<html>...</html>
```

Въпреки широкото си разпространение, HTTP/1.1 има няколко съществени ограничения, които влияят негативно на производителността на съвременните уеб приложения.

Първото и най-значимо ограничение е **head-of-line blocking**. При HTTP/1.1 заявките по една връзка се обработват последователно – сървърът трябва да изпрати

пълния отговор на първата заявка, преди да започне обработката на втората. Ако първата заявка е за голям ресурс, всички следващи заявки трябва да чакат, дори ако техните отговори са готови.

Второто ограничение е свързано с **текстовите хедъри**. HTTP/1.1 хедърите са текстови и се повтарят при всяка заявка. При съвременните уеб приложения, които изпращат десетки заявки за една страница, това води до значителен overhead. Cookies, User-Agent и други хедъри могат да заемат стотици байтове, които се предават отново и отново.

Третото ограничение е необходимостта от **множество TCP връзки** за постигане на паралелизъм. Браузърите обикновено отварят 6-8 паралелни връзки към един домейн, за да заобиколят head-of-line blocking. Това обаче води до допълнителен overhead от TCP handshakes и увеличена консумация на ресурси.

3.1.2 HTTP/2

HTTP/2, дефиниран в RFC 7540 [10] и актуализиран в RFC 9113 [11], е бинарен протокол, проектиран да адресира ограниченията на HTTP/1.1. Разработен първоначално от Google под името SPDY, HTTP/2 беше стандартизиран през 2015 година и днес се поддържа от всички съвременни браузъри и сървъри.

Ключовата иновация в HTTP/2 е **мултиплексирането**. Вместо да обработва заявките последователно, HTTP/2 позволява множество потоци (streams) да споделят една TCP връзка. Всеки поток има уникален идентификатор и може да пренася независима заявка-отговор двойка. Това елиминира head-of-line blocking на приложно ниво и позволява пълноценно използване на една TCP връзка.

HPACK компресията [12] е специализиран алгоритъм за компресия на HTTP хедъри. HPACK използва статична таблица с често срещани хедъри, динамична таблица за хедъри, специфични за връзката, и Huffman кодиране за стойностите. Това може да намали размера на хедърите с над 90% в сравнение с HTTP/1.1.

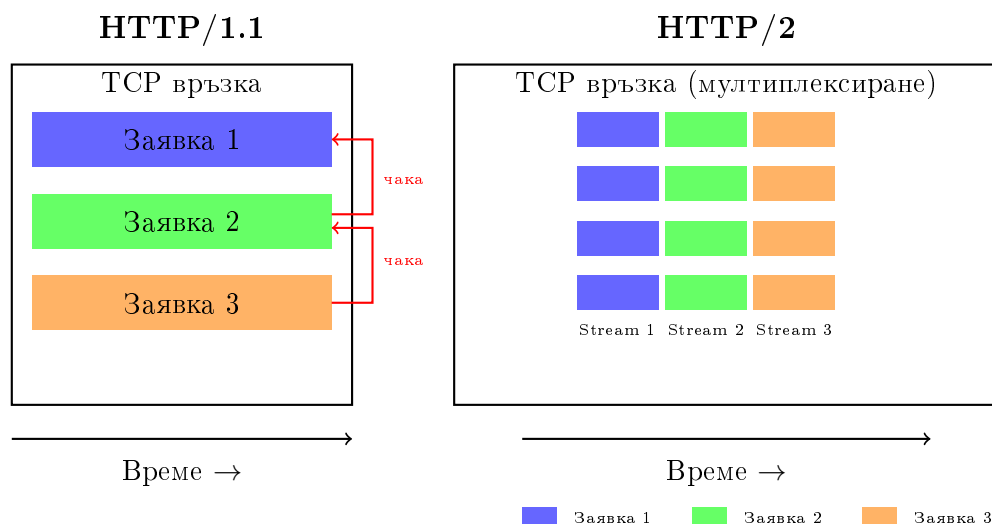
Server Push позволява на сървъра проактивно да изпраща ресурси към клиента, без клиентът да ги е заявил изрично. Например, когато клиентът заяви HTML страница, сървърът може едновременно да изпрати CSS и JavaScript файловете, които знае, че ще бъдат необходими. Това намалява латентността при зареждане на страници.

Приоритизацията позволява на клиента да указва относителната важност на различните потоци. Браузърът може да даде по-висок приоритет на CSS файловете (необходими за рендериране) спрямо изображенията (които могат да се заредят по-късно).

HTTP/2 използва фреймове като основна единица за комуникация. Всеки фрейм има фиксиран 9-байтов хедър, съдържащ дължина, тип, флагове и идентификатор на потока. Основните типове фреймове включват HEADERS за HTTP хедъри, DATA за тялото на съобщението, SETTINGS за конфигурационни

параметри, WINDOW_UPDATE за контрол на потока и GOAWAY за graceful затваряне на връзката.

Фигура 1 илюстрира ключовата разлика между HTTP/1.1 и HTTP/2 при обработка на множество заявки.



Фигура 1: Сравнение на HTTP/1.1 (последователна обработка с head-of-line blocking) и HTTP/2 (мултиплексиране на потоци)

3.2 WebSocket протокол

WebSocket [13] е протокол, проектиран за двупосочна комуникация в реално време върху една TCP връзка. Стандартизиран през 2011 година, WebSocket запълва важна празнина в уеб технологиите, позволявайки на сървъра да изпраща данни към клиента без клиентът да е направил изрична заявка.

Преди WebSocket, разработчиците използваха различни техники за симулиране на real-time комуникация, като polling (периодични заявки от клиента), long polling (заявки, които сървърът задържа докато има нови данни) и Server-Sent Events (еднопосочен поток от сървъра). Всички тези техники имат недостатъци – polling е неефективен, long polling създава сложност, а SSE е само еднопосочен.

WebSocket решава тези проблеми, като предоставя пълноценна двупосочна комуникация с минимален overhead. След първоначалния handshake, данните се предават като леки фреймове без HTTP хедъри, което значително намалява латентността.

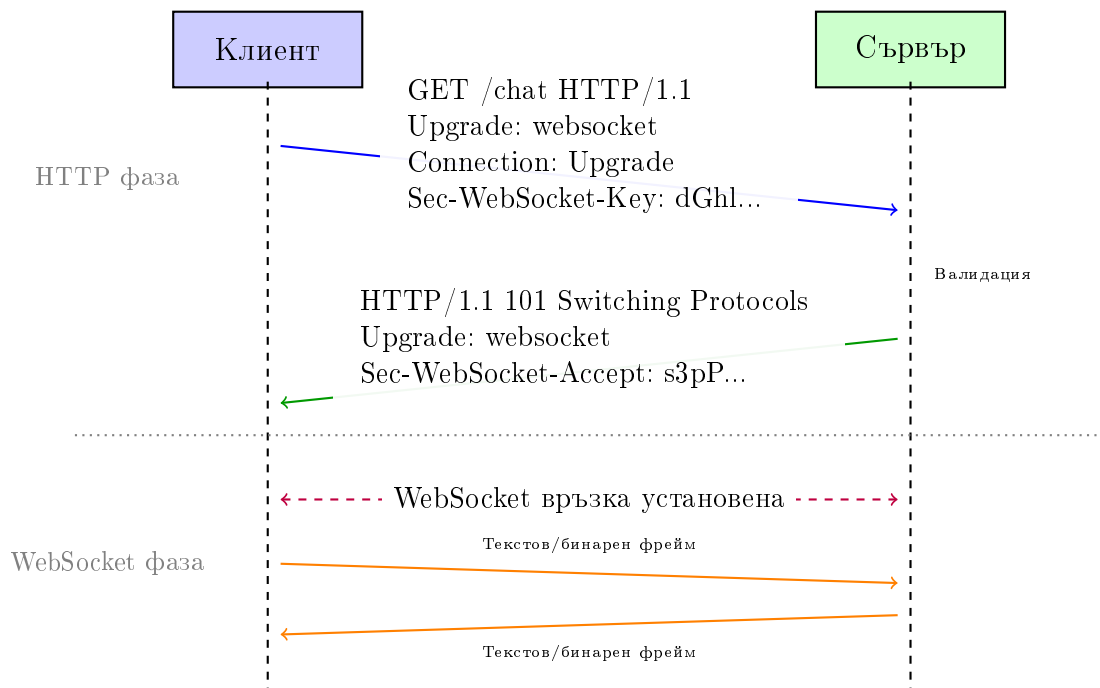
Процесът на установяване на WebSocket връзка започва с HTTP Upgrade заявка. Клиентът изпраща стандартна HTTP заявка с хедъри Upgrade: websocket и Connection: Upgrade, заедно с криптографски ключ в Sec-WebSocket-Key. Сървърът отговаря с HTTP 101 Switching Protocols и изчислен accept key, базиран на клиентския ключ и фиксиран GUID. След този handshake, TCP връзката преминава в WebSocket режим.

Заявка за upgrade изглежда по следния начин:

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

WebSocket съобщенията се предават като фреймове. Всеки фрейм има хедър, съдържащ опкод (тип на съобщението), флагове и информация за дължината. Опкодът определя как да се интерпретира payload-а: 0x1 за текстови данни (UTF-8), 0x2 за бинарни данни, 0x8 за затваряне на връзката, 0x9 за ping и 0xA за pong. Ping/pong механизмът се използва за keep-alive и за проверка дали връзката е все още активна.

Фигура 2 илюстрира процеса на WebSocket handshake.



Фигура 2: Sequence диаграма на WebSocket handshake процеса

Важна характеристика на WebSocket е маскирането на данните от клиента към сървъра. Всички фреймове, изпратени от клиента, трябва да бъдат маскирани с 4-байтов ключ. Това е мярка за сигурност, предотвратяваща определени видове атаки срещу прокси сървъри.

3.3 TLS и сигурност

Transport Layer Security (TLS) е криптографски протокол, осигуряващ сигурна комуникация през интернет. TLS е наследник на SSL (Secure Sockets Layer) и е стандартът за криптиране на уеб трафик. Когато URL адрес започва с "https://",

това означава, че връзката е защитена с TLS.

TLS предоставя три основни гаранции за сигурност. Първо, **конфиденциалност** – данните са криптирани и не могат да бъдат прочетени от трети страни. Второ, **интегритет** – данните не могат да бъдат модифицирани незабелязано по време на предаване. Трето, **автентикация** – клиентът може да верифицира идентичността на сървъра чрез цифрови сертификати.

TLS 1.3 [14], публикуван през 2018 година, въвежда значителни подобрения спрямо предходните версии. Handshake процесът е опростен и изисква само един round-trip (1-RTT) за установяване на връзка, в сравнение с два round-trips при TLS 1.2. За клиенти, които вече са се свързвали със сървъра, е възможен 0-RTT режим, при който данни могат да се изпращат още с първия пакет.

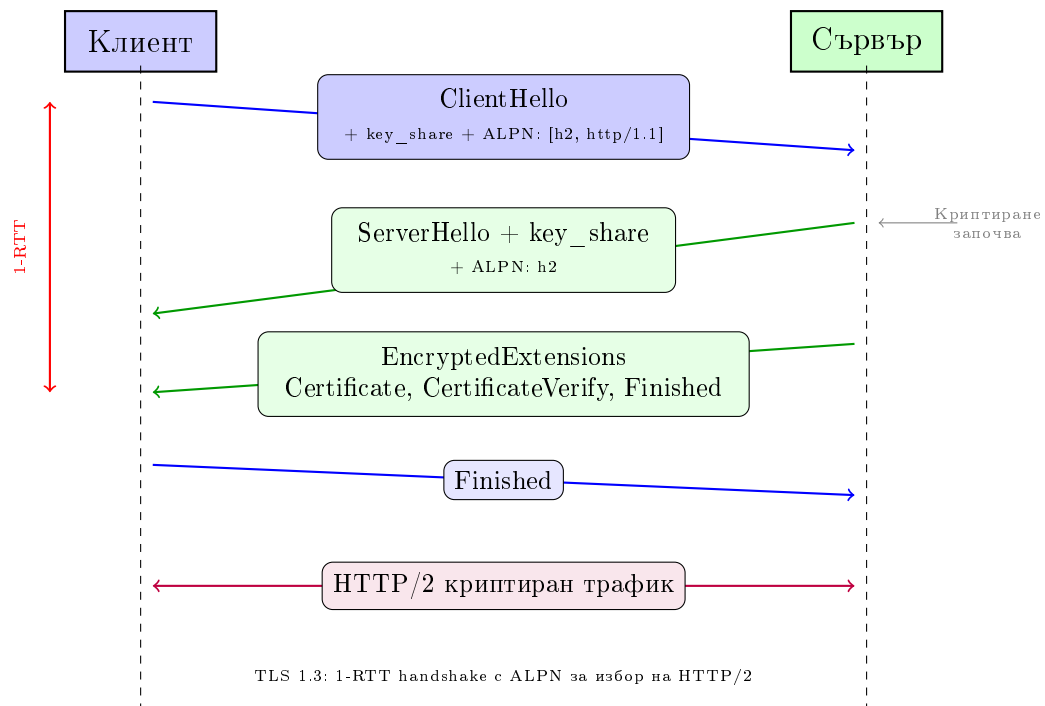
TLS 1.3 премахва поддръжката за остарели и несигурни криптографски алгоритми. Поддържат се само съвременни cipher suites, базирани на AEAD (Authenticated Encryption with Associated Data) алгоритми като AES-GCM и ChaCha20-Poly1305. Това опростява конфигурацията и елиминира възможността за downgrade атаки.

ALPN (Application-Layer Protocol Negotiation) [15] е TLS разширение, което позволява избор на приложен протокол по време на handshake. Това е механизмът, чрез който HTTP/2 се негоцира при TLS връзки.

При ALPN негоциация, клиентът включва в ClientHello съобщението списък с поддържани протоколи, подредени по предпочитание. Например, съвременен браузър би изпратил “h2” (HTTP/2) и “http/1.1” като опции. Сървърът избира един от предложените протоколи (обикновено най-предпочитания, който поддържа) и го включва в ServerHello отговора. След завършване на TLS handshake, комуникацията продължава с избрания протокол.

ALPN е критичен за HTTP/2, тъй като позволява на клиента и сървъра да се договорят за протокола преди да започнат да обменят HTTP данни. Без ALPN, би било необходимо допълнително round-trip за upgrade от HTTP/1.1 към HTTP/2.

Фигура 3 илюстрира TLS 1.3 handshake процеса с ALPN негоциация.



Фигура 3: TLS 1.3 handshake с ALPN негоциация за HTTP/2

3.4 C++20 корутини

Корутините са една от най-значимите нови функционалности в C++20 [2]. Те представляват обобщение на концепцията за функция, позволявайки на функцията да спре изпълнението си в определени точки и да го възобнови по-късно, без да губи своето състояние.

Традиционните функции следват модела “call/return” – когато функция бъде извикана, тя се изпълнява до край и връща резултат. Корутините добавят възможността за “suspend/resume” – функцията може да спре изпълнението си, да върне контрола на извикващия, и по-късно да продължи от точката, където е спряла.

Тази възможност е изключително полезна за асинхронно програмиране. Вместо да използваме callbacks или promises, можем да пишем код, който изглежда синхронен и последователен, но всъщност се изпълнява асинхронно. Когато корутината срещне операция, която не може да завърши веднага (например четене от мрежата), тя спира и позволява на други корутини да се изпълняват. Когато операцията завърши, корутината се възобновява и продължава изпълнението си.

C++20 въвежда три нови ключови думи за работа с корутини. `co_await` се използва за спиране на корутината докато чака завършване на асинхронна операция. `co_yield` се използва за генериране на стойност и временно спиране (полезно за генератори). `co_return` се използва за връщане на крайния резултат и завършване на корутината.

3.4.1 Основни концепции

За да разберем как работят корутините в C++20, трябва да се запознаем с няколко ключови концепции: `promise type`, `awaitable` и `coroutine handle`.

Promise type е тип, който контролира поведението на корутината. Когато компилаторът срещне функция, съдържаща `co_await`, `co_yield` или `co_return`, той автоматично генерира код, който използва `promise type` за управление на корутината. `Promise type` определя какво се случва при стартиране на корутината, при нейното завършване, при връщане на стойност и при възникване на изключение.

```
1 struct Promise {
2     Task<T> get_return_object();
3     std::suspend_always initial_suspend();
4     FinalAwaiter final_suspend() noexcept;
5     void return_value(T value);
6     void unhandled_exception();
7 };
```

Listing 1: `Promise type` структурата

Методът `get_return_object()` създава обекта, който се връща на извикващия при създаване на корутината. Методът `initial_suspend()` определя дали корутината да започне изпълнение веднага или да изчака. Методът `final_suspend()` определя какво се случва след завършване на корутината – дали да се унищожи автоматично или да остане в паметта.

Awaitable е тип, който може да се използва с оператора `co_await`. `Awaitable` обектите представляват асинхронни операции, които могат да спрат корутината докато чакат завършване.

```
1 struct Awaitable {
2     bool await_ready();
3     void await_suspend(std::coroutine_handle<>);
4     T await_resume();
5 };
```

Listing 2: `Awaitable` интерфейс

Методът `await_ready()` връща `true`, ако операцията вече е завършила и не е необходимо спиране. Методът `await_suspend()` се извиква, когато корутината трябва да спре – тук се регистрира `callback` за възобновяване. Методът `await_resume()` връща резултата от операцията, когато корутината се възобнови.

Coroutine handle е низкониво представяне на корутината, подобно на указател. Чрез `handle` можем да възобновим спряна корутина, да проверим дали е завършила, или да я унищожим. `Handle`-ът се използва вътрешно от `awaitable` обектите за управление на корутините.

3.4.2 Предимства на корутините

Корутините предоставят множество предимства пред традиционните подходи за асинхронно програмиране.

Първото и най-важно предимство е **четимостта**. Асинхронният код, написан с корутини, изглежда почти идентично на синхронен код. Вместо вложени callbacks или сложни promise chains, имаме последователни изрази, разделени с `co_await`. Това значително намалява когнитивното натоварване при четене и писане на асинхронен код.

Второто предимство е **ефективността**. За разлика от нишките, корутините не изискват системни извиквания за създаване или превключване. Превключването между корутини е просто запазване и възстановяване на няколко регистъра – операция, която отнема наносекунди вместо микросекунди.

Третото предимство е **композируемостта**. Корутините могат свободно да се влагат една в друга. Една корутина може да извика друга корутина с `co_await`, която от своя страна може да извика трета. Това позволява изграждане на сложна асинхронна логика от прости градивни блокове.

Четвъртото предимство е, че корутините са **stackless**. Традиционните нишки изискват отделен стек (обикновено 1MB), който се заделя при създаване. Корутините съхраняват само локалните променливи и точката на изпълнение – типично десетки до стотици байтове. Това позволява създаване на милиони едновременни корутини на машина с ограничена памет.

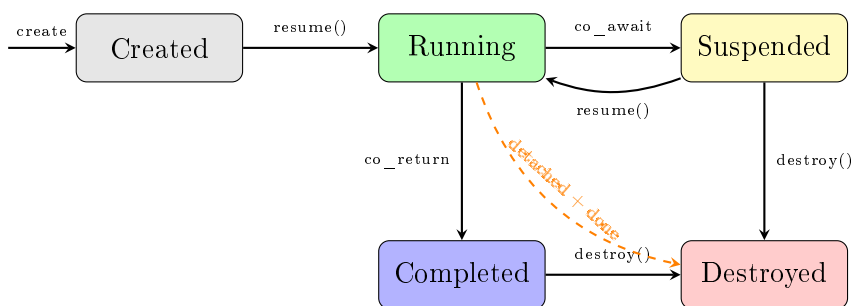
Следният пример демонстрира как асинхронен код с корутини изглежда почти идентично на синхронен:

```
1 Task<Response> handle_request(Request& req) {
2     // Всяка от тези операции може да отнеме време,
3     // но кодът изглежда последователен
4     auto user = co_await db.find_user(req.param("id"));
5     auto posts = co_await db.get_posts(user.id);
6     co_return render_template("profile", user, posts);
7 }
```

Listing 3: Пример за асинхронен код с корутини

В този пример, докато `db.find_user()` чака отговор от базата данни, корутината е спряна и нишката може да обслужва други заявки. Когато отговорът пристигне, корутината се възобновява и продължава със следващия ред.

Фигура 4 илюстрира жизнения цикъл на корутина и възможните преходи между състоянията.



Пунктирна линия: detached корутина се самоунищожава

Фигура 4: State диаграма на жизнения цикъл на корутина

3.5 Асинхронен I/O

Асинхронният вход/изход (I/O) е фундаментална концепция за изграждане на високопроизводителни мрежови приложения. При синхронен I/O, когато програмата извърши операция като четене от сокет, тя блокира докато данните пристигнат. При асинхронен I/O, програмата иницира операцията и продължава изпълнението си, като бива уведомена когато операцията завърши.

Различните операционни системи предоставят различни механизми за асинхронен I/O. Всеки от тях има своите характеристики и оптимални сценарии на използване. За постигане на максимална производителност, Coroute използва native механизма на всяка платформа.

3.5.1 IOCP (Windows)

I/O Completion Ports (IOCP) [16] е механизмът на Windows за мащабируем асинхронен I/O. IOCP е проектиран специално за сървърни приложения, които трябва да обслужват хиляди едновременни връзки.

При IOCP, приложението създава completion port – обект на ядрото, който служи като централна точка за събиране на завършени I/O операции. Когато приложението иска да извърши асинхронна операция (например четене от сокет), то подава заявка към операционната система и продължава изпълнението си. Когато операцията завърши, резултатът се поставя в completion port.

Работни нишки извикват `GetQueuedCompletionStatus()`, за да извлекат завършени операции от completion port. Ако няма готови операции, нишката блокира ефективно, без да консумира CPU ресурси. Windows автоматично управлява броя на активните нишки, за да оптимизира използването на CPU.

Едно от ключовите предимства на IOCP е неговата мащабируемост. Малък брой нишки (обикновено равен на броя на CPU ядрата) може да обслужва десетки хиляди едновременни връзки. Това е възможно, защото нишките не блокират при отделни I/O операции, а само при чакане за завършени операции.

В Coroute, имплементацията на IOCP използва няколко ключови техники за максимална производителност. За трансфер на файлове се използва нейтив

функцията `TransmitFile`, която позволява директен zero-copу трансфер от файловата система към мрежовия сокет. Използва се и механизма на `OVERLAPPED` структурите за ефективно проследяване на състоянията на асинхронните операции, което премахва нуждата от сложна синхронизация при обработка на завършените събития.

3.5.2 `io_uring` (Linux)

`io_uring` [17] е модерен Linux интерфейс за асинхронен I/O, въведен в ядро версия 5.1 (2019). Той е проектиран да замени по-старите интерфейси като `epoll` и `aio`, предоставяйки по-висока производителност и по-богата функционалност.

Архитектурата на `io_uring` се базира на две кръгови опашки (ring buffers), споделени между потребителското пространство и ядрото. Submission Queue (SQ) се използва от приложението за подаване на I/O заявки. Completion Queue (CQ) се използва от ядрото за връщане на резултати от завършени операции.

Ключово предимство на `io_uring` е възможността за batching на системни извиквания. Вместо да прави отделно системно извикване за всяка I/O операция, приложението може да добави множество заявки в SQ и да ги подаде наведнъж с едно извикване на `io_uring_enter()`. Това значително намалява overhead-a от превключване между потребителско пространство и ядро.

`io_uring` поддържа и zero-copу операции, при които данните се прехвърлят директно между мрежовия буфер и файловата система, без копиране в потребителското пространство. Това е особено полезно за файлови сървъри и CDN приложения.

Имплементацията на `io_uring` в Coroute е проектирана за изключителна производителност чрез архитектура с **per-thread rings**. Всяка нишка разполага със собствен `io_uring` инстанс, което елиминира конкуренцията за заключвания (lock contention) между нишките. При работа като сървър се използва `SO_REUSEPORT` за балансиране на входящите връзки директно от ядрото между отделните нишки (**multi-accept**). Обработката на завършените операции се извършва на партиди (**batching**), като се използва нейтив функцията `sendfile` за zero-copу производителност при работа с файлове.

3.5.3 `kqueue` (macOS/BSD)

`kqueue` [18] е механизъм за event notification, наличен в macOS и BSD операционните системи. Въведен първоначално във FreeBSD 4.1 (2000), `kqueue` предоставя унифициран интерфейс за наблюдение на различни типове събития.

При `kqueue`, приложението регистрира интерес към определени събития чрез структури, наречени kevents. Събитията могат да бъдат свързани с файлови дескриптори (готовност за четене/писане), сигнали, таймери, промени във файловата система и други. След регистрация, приложението извиква `kevent()` за да получи списък с настъпили събития.

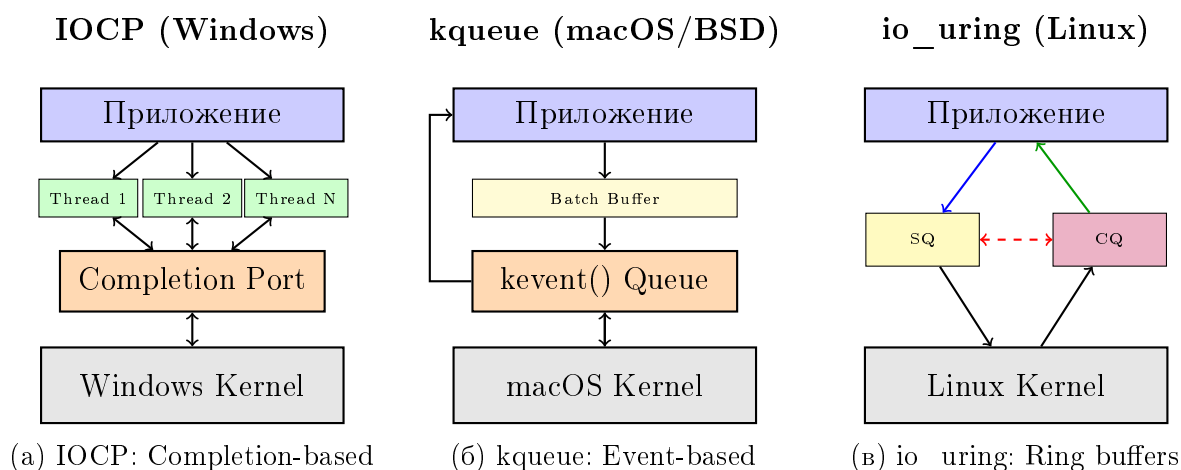
Едно от предимствата на `kqueue` е неговата гъвкавост. За разлика от `epoll`, който работи само с файлови дескриптори, `kqueue` може да наблюдава разнообразни

типове събития. Това опростява архитектурата на приложения, които трябва да реагират на множество видове асинхронни събития.

kqueue също поддържа edge-triggered и level-triggered режими, подобно на epoll. В edge-triggered режим, събитие се докладва само когато състоянието се промени. В level-triggered режим, събитие се докладва докато условието е изпълнено.

В Coroute, имплементацията на kqueue е оптимизирана за максимална производителност чрез няколко ключови техники. Първо, използва се **syscall batching**, при който множество регистрации на събития се натрупват в thread-local буфер и се подават на ядрото заедно с извикването за изчакване на нови събития. Това елиминира излишните системни извиквания за всяка асинхронна операция. Второ, използван е **lock-free** подход за проследяване на чакащите операции чрез **udata** полето на **kevent** структурите, което премахва нуждата от глобални заключвания при обработка на събития. Трето, за пренос на файлове се използва нейтив функцията **sendfile**, която осигурява zero-copy трансфер на данни директно от файловата система към мрежовия сокет.

Фигура 5 сравнява архитектурите на IOCP, kqueue и io_uring.



Фигура 5: Сравнение на архитектурите на асинхронния I/O при различните платформи

3.6 DFA-базирано маршрутизиране

Маршрутизирането е процесът на определяне кой handler да обработи дадена HTTP заявка, базирано на URL адреса и HTTP метода. В типично уеб приложение може да има стотици или дори хиляди различни маршрути, всеки с различен URL шаблон.

Традиционният подход за маршрутизиране е последователно проверяване на всеки шаблон срещу входния URL. При този подход, за всяка входяща заявка се итерира през списъка с маршрути и се проверява дали URL-ът съответства на шаблона. Това води до времева сложност $O(N \times M)$, където N е броят на маршрутите, а M е дължината на URL-а. При голям брой маршрути, това може да стане значително забавяне.

В научната публикация „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1] е представен алтернативен подход, базиран на детерминиран краен автомат (DFA). Този алгоритъм позволява съпоставяне на входен текст с множество регулярни изрази едновременно, с линейна времева сложност спрямо дължината на входа.

3.6.1 Теоретична основа

За да разберем DFA-базираното маршрутизиране, трябва първо да се запознаем с теорията на формалните езици и автомати.

Регулярните изрази и крайните автомати са еквивалентни по изразителна сила – това е известно като теорема на Kleene. Всеки регулярен израз може да се преобразува в еквивалентен недетерминиран краен автомат (NFA), който от своя страна може да се преобразува в детерминиран краен автомат (DFA) [19, 20].

Краен автомат M се дефинира формално като петорка $(Q, \Sigma, \delta, q_0, F)$, където Q е крайно множество от състояния, Σ е входната азбука (множеството от възможни символи), $\delta : Q \times \Sigma \rightarrow Q$ е функцията на преходите, $q_0 \in Q$ е началното състояние, и $F \subseteq Q$ е множеството от крайни (приемащи) състояния.

При обработка на входен низ, автоматът започва от началното състояние q_0 . За всеки символ от входа, автоматът преминава в ново състояние, определено от функцията на преходите δ . Ако след обработка на целия вход автоматът е в крайно състояние от F , входът се приема; в противен случай се отхвърля.

Ключовото предимство на DFA е, че за всяка комбинация от текущо състояние и входен символ има точно един възможен преход. Това означава, че обработката на входа изисква точно толкова стъпки, колкото е дължината на входа – линейна времева сложност $O(N)$.

3.6.2 Алгоритъм за изграждане на pattern graph

Алгоритъмът от „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1] разширява класическия подход, като комбинира множество регулярни изрази в един обединен граф, наречен pattern graph. Този граф е подобен на trie структура, но с възможност за цикли, които са необходими за представяне на операторите за повторение ($*$, $+$, $\{n,m\}$).

Процесът на изграждане започва с празен граф, съдържащ само начално състояние. За всеки регулярен израз (URL шаблон), алгоритъмът добавя съответните състояния и преходи към графа. Когато два шаблона споделят общ префикс, те споделят и съответните състояния в графа. Това е ключът към ефективността – вместо да проверяваме всеки шаблон поотделно, ние обхождаме един общ граф.

Всяко крайно състояние в графа е асоциирано с един или повече маршрути. Когато входният URL достигне крайно състояние, знаем кой маршрут е съвпаднал. Ако множество маршрути съвпадат, се използва приоритет (обикновено по-специфичният маршрут има предимство).

Времовата сложност на алгоритъма е следната: предобработката (изграждане

на графа) изисква време $O(\sum \text{regex_length})$, пропорционално на общата дължина на всички шаблони. Търсенето (съпоставяне на URL) изисква време $O(N)$, където n е дължината на входния URL. Забележете, че времето за търсене не зависи от броя на маршрутите – това е ключовото подобрение спрямо наивния подход.

3.6.3 Приложение в Coroute

В контекста на Coroute, URL шаблоните се преобразуват в регулярни изрази. Например, шаблонът `/users/{id}` се преобразува в регулярен израз, който съвпада с `/users/` последвано от произволна последователност от символи (параметъра `id`).

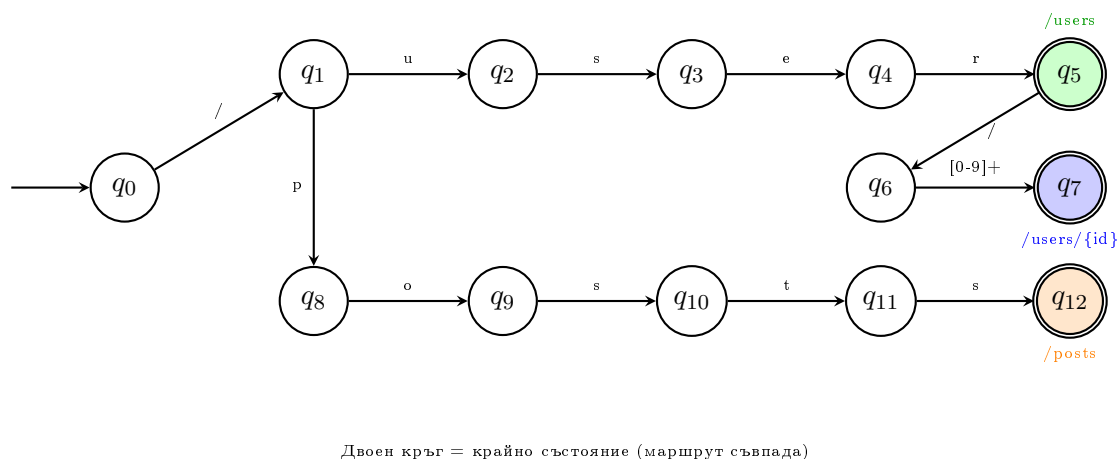
При стартиране на сървъра, всички регистрирани маршрути се компилират в един DFA. При всяка входяща заявка, URL-ът се обработва от DFA с една линейна обиколка. Резултатът е или съвпадение с конкретен маршрут (заедно с извлечените параметри), или липса на съвпадение (404 Not Found).

3.6.4 Benchmark резултати

Според експерименталните резултати от „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1], предложеният алгоритъм показва значително подобрение в производителността спрямо традиционните подходи. При тестове със 100 различни регулярни изрази, алгоритъмът е до **62.25 пъти по-бърз** от последователното съпоставяне с `std::regex`.

Това подобрение е особено значимо за уеб приложения с голям брой маршрути. При REST API с ресурси, версии и вложени пътища, броят на маршрутите лесно може да достигне стотици. С DFA-базирано маршрутизиране, времето за намиране на правилния handler остава константно, независимо от броя на маршрутите.

Фигура 6 илюстрира пример за DFA, който разпознава URL шаблоните `/users`, `/users/{id}` и `/posts`.



Фигура 6: Пример за DFA за URL маршрутизиране с три шаблона

3.7 Foreign Function Interface (FFI)

В съвременната софтуерна разработка често възниква необходимост от комбиниране на код, написан на различни езици за програмиране. Механизмът, който

позволява на програма, написана на един език, да извиква подпрограми или да използва услуги, написани на друг език, се нарича Foreign Function Interface (FFI).

Исторически, C езикът се е утвърдил като *de facto* стандарт за системна оперативна съвместимост. Почти всички съвременни езици от високо ниво (Python, Java, Dart, Rust, Go) предоставят възможност за извикване на C функции чрез съответните FFI имплементации (напр. JNI за Java, `ctypes` за Python, или `dart:ffi` за Dart). Тъй като C++ поддържа експортиране на функции със C свързаност (чрез ключовата дума `extern "C"`), той може напълно да се интегрира в тези екосистеми.

3.7.1 Паметови граници и управление

При преминаване на границата между два езика чрез FFI, основното предизвикателство е управлението на паметта. Езиците с автоматично събиране на боклука (Garbage Collection), като Dart, имат строги правила за локация и преместване на обекти в паметта. C++, от друга страна, очаква ръчно управление на паметта. Затова данните, които се предават през FFI, обикновено изискват алокиране на непрекъснати блокове памет извън хийпа на garbage collector-а или внимателно синхронизиране на жизнения цикъл на обектите.

3.7.2 Контекст на изпълнение и асинхронност

Извикването на C/C++ функция от език на по-високо ниво обикновено блокира текущата нишка до завършване на функцията. Това е проблематично в среди като Dart (Flutter), където потребителският интерфейс работи в единна основна нишка (isolate). За да не блокира UI-а, дългите операции в C++ трябва да се изпълняват в системни или background нишки. Възниква необходимост от механизъм за обратна връзка (callbacks), чрез който C++ може асинхронно да уведоми хост езика за завършване на операция, подавайки съобщение обратно през границата на езиците. Този модел е ключов за изграждане на "headless" двигатели, където бизнес логиката и I/O операциите се изпълняват ефективно от C++, докато фронтендът само реагира на събития.

3.8 Шаблонни машини (Template Engines)

Шаблонните машини са софтуерни компоненти, които комбинират статичен шаблон с динамични данни за генериране на крайни документи (обикновено HTML, XML или други текстови формати). В контекста на уеб приложенията, те са фундаментална част от архитектурата Server-Side Rendering (SSR).

Разделянето на презентационния слой от бизнес логиката е основен принцип в шаблонизирането. Шаблонът обикновено съдържа крайния маркъп, в който са вмъкнати специални тагове или директиви за контрол (напр. цикли, условни конструкции, променливи). По време на изпълнение, шаблонната машина парсва тези файлове, оценява контролните директиви спрямо подаден контекст (речник с данни) и синтезира крайния низ.

Имплементации като Jinja (C++ имплементация, базирана на популярния Jinja синтаксис за Python), използват JSON като формат за подаване на контекстуални данни. Интеграцията им директно в уеб сървър позволява генерирането на персонализирани отговори без необходимост от допълнителни мрежови заявки от клиента към API, което намалява латентността при първоначално зареждане.

4 Архитектура на Coroute

След като разгледахме теоретичните основи в предходната глава, сега ще представим конкретната архитектура на библиотеката Coroute. Тази глава описва модулната структура на библиотеката, жизнения цикъл на HTTP заявките, корутинния модел на изпълнение, middleware системата и платформено-независимия I/O слой.

Архитектурата на Coroute е проектирана с няколко ключови принципа. Първо, **модулност** – всеки компонент има ясно дефинирана отговорност и може да се използва независимо. Второ, **разширяемост** – новите протоколи и функционалности могат да се добавят без промяна на съществуващия код. Трето, **ефективност** – архитектурата минимизира копирането на данни и системните извиквания. Четвърто, **безопасност** – типовата система на C++ се използва за предотвратяване на грешки по време на компилация.

4.1 Обща архитектура и модулна структура

Coroute е организиран в йерархия от модули, всеки от които отговаря за определен аспект на функционалността. Тази организация улеснява разбирането на кода, тестването и поддръжката.

Модулът **core/** съдържа основните компоненти на библиотеката. Класът **App** е централната точка за конфигуриране и стартиране на сървъра. **Router** управлява маршрутизирането на заявките към съответните handlers. Класовете **Request** и **Response** представляват HTTP заявки и отговори с удобен API за достъп до хедъри, тяло и параметри.

Модулът **coro/** предоставя корутинната инфраструктура. Типът **Task<T>** е основният тип за асинхронни операции. **CancellationToken** позволява кооперативно прекратяване на дълготрайни операции. Тук се намират и различните **awaiter** типове за интеграция с I/O операциите.

Модулът **net/** съдържа мрежовия слой. **IoContext** е абстракция над платформено-специфичните I/O механизми. **Connection** представлява TCP връзка с методи за асинхронно четене и писане. Тук се намират и имплементациите за TLS и WebSocket.

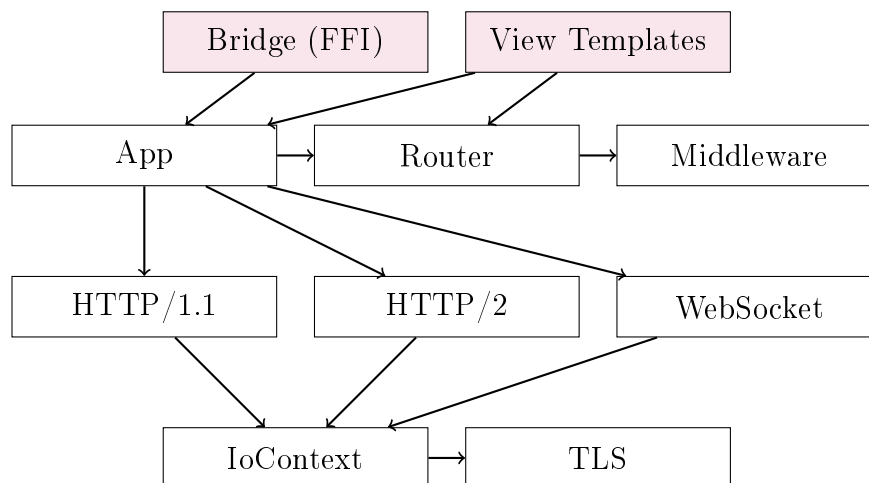
Модулът **http2/** съдържа пълната HTTP/2 имплементация. **Http2Connection** управлява HTTP/2 връзка с множество потоци. HPACK encoder/decoder се грижи за компресията на хедъри. **Frame parser/serializer** обработва бинарните HTTP/2 фреймове.

Модулът **util/** съдържа помощни функции и типове. **expected<T, E>** е тип за представяне на резултат или грешка (подобен на **std::expected** от C++23). **FromString<T>** е trait за конвертиране на низове към типове. Тук се намират и функциите за zero-copy file transfer.

Модулът **bridge/** предоставя C/C++ FFI границата за експониране на вътрешностите на Coroute към хост среди като Flutter. Този модул дефинира плосък

С API интерфейс, изолиращ C++ изключенията и управляващ жизнения цикъл на корутините при комуникация с външни runtime среди.

Модулът **view/** инкапсулира логиката за сървърно рендиране на шаблони. Включва структури като **ViewTemplates** за компилация и кеширане на Inja шаблони, **ViewRouteInfo** за конфигурация на изгледи и интеграция с **ResponseBuilder** за конструиране на отговори.



Фигура 7: Модулна архитектура на Coroute

4.2 Жизнен цикъл на HTTP заявка

Разбирането на жизнения цикъл на HTTP заявката е ключово за ефективното използване на Coroute. Всяка заявка преминава през поредица от етапи, като всеки етап се изпълнява асинхронно чрез корутини.

Първият етап е **Accept**. **IoContext** наблюдава listening socket-а за входящи TCP връзки. Когато клиент се свърже, се създава нов **Connection** обект и се стартира корутина за обработка на връзката. Тази корутина работи в режим “detached” – тя се самоуправлява и се унищожава автоматично при завършване.

Вторият етап е **TLS Handshake**, който е опционален и се изпълнява само ако сървърът е конфигуриран за HTTPS. По време на handshake се установява криптирана връзка и се верифицира сертификатът на сървъра. TLS handshake също е асинхронен – корутината спира докато чака завършване на криптографските операции.

Третият етап е **ALPN Negotiation**, който се случва като част от TLS handshake. Клиентът и сървърът се договарят за протокола – HTTP/2 (“h2”) или HTTP/1.1 (“http/1.1”). Ако клиентът поддържа HTTP/2 и сървърът е конфигуриран за него, се избира HTTP/2. В противен случай се използва HTTP/1.1.

Четвъртият етап е **Parse**. Сървърът чете данни от връзката и парсва HTTP заявката. При HTTP/1.1 това включва парсане на request line, хедъри и тяло. При HTTP/2 се декодират бинарни фреймове и HPACK-компресирани хедъри.

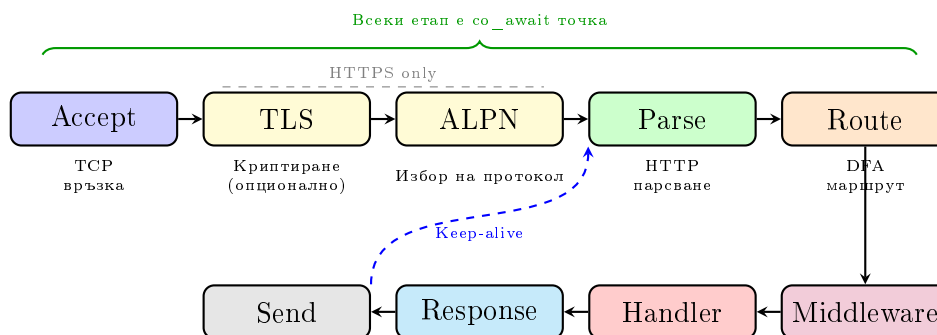
Петият етап е **Route**. DFA matcher-ът съпоставя URL-а на заявката с регистрираните маршрути. Ако има съвпадение, се извличат параметрите от URL-а. Ако няма съвпадение, се връща 404 Not Found.

Шестият етап е **Middleware**. Заявката преминава през веригата от middleware компоненти. Всеки middleware може да модифицира заявката, да върне отговор директно, или да предаде контрола на следващия middleware.

Седмият етап е **Handler**. Потребителският handler се извиква с заявката и параметрите. Handler-ът е корутина, която може да извършва асинхронни операции като заявки към база данни или външни API-та.

Осмият етап е **Response**. Отговорът се сериализира и изпраща към клиента. При HTTP/1.1 това е текстов формат. При HTTP/2 отговорът се разбива на фреймове и хедърите се компресират с HPACK.

Фигура 8 илюстрира пълния жизнен цикъл на HTTP заявка.



Фигура 8: Жизнен цикъл на HTTP заявка в Coroute

Следният код от `app.cpp` илюстрира основния accept loop:

```

1 [this]() -> Task<void> {
2     while (!cancel_source_.is_cancelled()) {
3         auto conn_result = co_await listener_->async_accept();
4         if (!conn_result) {
5             if (cancel_source_.is_cancelled()) break;
6             continue;
7         }
8         handle_connection(std::move(*conn_result)).start_detached();
9     }
10 }().start_detached();

```

Listing 4: Основен accept loop

Методът `start_detached()` стартира корутината в режим “fire-and-forget” – тя се самоунищожава при завършване.

4.3 Корутинен модел на изпълнение – Task<T>

Централен елемент на Coroute е типът Task<T>, който представлява асинхронна операция, връщаща стойност от тип T. Този тип е сърцето на корутинната инфраструктура и определя как корутините се създават, изпълняват и завършват.

Task<T> е проектиран да бъде “lazy” – корутината не започва изпълнение при създаване, а чак когато бъде awaited. Това позволява композиране на корутини без нежелани странични ефекти. Също така, Task<T> поддържа режим “detached”, при който корутината се самоуправлява и се унищожава автоматично при завършване.

4.3.1 Promise Type

Всяка корутина в C++ има асоцииран promise_type, който контролира нейното поведение. Promise type-ът определя какво се случва при различните етапи от живота на корутината – създаване, спиране, възобновяване и завършване.

```
1 struct TaskPromiseBase {
2     std::coroutine_handle<> continuation_ = std::noop_coroutine();
3     std::exception_ptr exception_;
4     CancellationToken cancel_token_;
5     bool detached_ = false;
6
7     std::suspend_always initial_suspend() noexcept { return {}; }
8     FinalAwaiter final_suspend() noexcept { return {}; }
9
10    void unhandled_exception() noexcept {
11        exception_ = std::current_exception();
12    }
13};
```

Listing 5: TaskPromiseBase структура

Ключови характеристики:

- initial_suspend() връща suspend_always – корутината е “lazy” и не започва изпълнение докато не бъде awaited
- continuation_ съхранява handle към извикващата корутина за възобновяване
- detached_ флаг указва дали корутината трябва да се самоунищожи

4.3.2 FinalAwaiter

При завършване на корутината, FinalAwaiter решава какво да се случи:

```
1 struct FinalAwaiter {
2     template<typename Promise>
3     std::coroutine_handle<> await_suspend(
4         std::coroutine_handle<Promise> h) noexcept {
5         auto& promise = h.promise();
```

```

6
7     // If detached, destroy ourselves
8     if (promise.detached_) {
9         h.destroy();
10        return std::noop_coroutine();
11    }
12
13    // Resume continuation if exists
14    if (promise.continuation_) {
15        return promise.continuation_;
16    }
17    return std::noop_coroutine();
18 }
19 };

```

Listing 6: FinalAwaiter логика

4.3.3 Awaiter

Когато една корутина “await-ва” `Task<T>`, се използва следният `awaiter`:

```

1 struct Awaiter {
2     handle_type handle_;
3
4     bool await_ready() const noexcept {
5         return !handle_ || handle_.done();
6     }
7
8     std::coroutine_handle<> await_suspend(
9         std::coroutine_handle<> continuation) noexcept {
10        handle_.promise().continuation_ = continuation;
11        return handle_; // Transfer control to awaited task
12    }
13
14    T await_resume() {
15        return std::move(handle_.promise()).result();
16    }
17 };

```

Listing 7: Task Awaiter

4.4 Безопасност на паметта при `detached` корутини

Използването на `start_detached()` въвежда специфични предизвикателства за управление на паметта. Когато корутина работи в `detached` режим, тя няма “собственик” — никой не чака нейното завършване и никой не държи `reference` към нея. Това създава риск от `use-after-free` грешки, ако корутината достъпва ресурси, които са били унищожени.

4.4.1 Проблемът

Разгледайте следния опасен код:

```

1 void dangerous_example() {
2     std::string local_data = "important";
3
4     [&local_data]() -> Task<void> {
5         co_await async_delay(100ms);
6         // DANGER: local_data may be destroyed!
7         std::cout << local_data << std::endl;
8     }().start_detached();
9 } // local_data is destroyed here

```

Listing 8: Опасен pattern с detached корутина

Когато `dangerous_example()` завърши, `local_data` се унищожава, но detached корутината все още може да работи и да се опита да достъпи унищожената памет.

4.4.2 Решения в Coroute

Coroute използва няколко техники за предотвратяване на тези проблеми:

1. Move семантика за Connection:

```

1 handle_connection(std::move(*conn_result)).start_detached();

```

Listing 9: Безопасно предаване на ownership

Connection обектът се `move`-ва в корутината, която поема пълната отговорност за неговия живот. Корутина унищожава connection-а при завършване.

2. RAII guards за ресурси:

```

1 Task<void> handle_connection(std::unique_ptr<Connection> conn) {
2     // conn is owned by this coroutine
3     // It will be destroyed when coroutine completes
4
5     while (!cancel_token.is_cancelled()) {
6         auto data = co_await conn->async_read(...);
7         if (!data) break;
8         // process...
9     }
10
11     conn->close(); // Explicit cleanup
12 } // conn destroyed here automatically

```

Listing 10: RAII pattern за connection lifecycle

3. Shared ownership когато е необходимо:

```

1 auto shared_state = std::make_shared<SessionState>();
2
3 [shared_state]() -> Task<void> {
4     // shared_state stays alive as long as coroutine runs
5     co_await process_session(shared_state);
6 }().start_detached();

```

Listing 11: Shared ownership за споделени ресурси

4. CancellationToken за кооперативно прекратяване:

При shutdown, сървърът сигнализира на всички активни корутини да завършат чрез CancellationToken. Това позволява graceful cleanup вместо насилствено прекратяване.

4.4.3 Правила за безопасност

При работа с detached корутини в Coroute, следвайте тези правила:

1. Никога не capture-вайте локални променливи по reference в detached корутина
2. Използвайте `std::move()` за прехвърляне на ownership
3. За споделени ресурси използвайте `std::shared_ptr`
4. Винаги проверявайте CancellationToken в дълготрайни операции

4.5 DFA-базирано маршрутизиране

Coroute използва алгоритъма, описан в „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1], за ефективно маршрутизиране на заявки.

4.5.1 Регистрация на маршрути

Маршрутите се регистрират чрез методите `get()`, `post()`, и др.:

```

1 app.get("/users/{id}", [](int id, Request& req) -> Task<Response> {
2     co_return Response::ok("User: " + std::to_string(id));
3 });
4
5 app.post("/api/items", [](Request& req) -> Task<Response> {
6     auto body = req.body();
7     co_return Response::created(body);
8 });

```

Listing 12: Регистрация на маршрути

4.5.2 Конвертиране на шаблони

Шаблоните се конвертират в регулярни изрази:

```

1 // /users/{id} -> /users/([A-Za-z0-9_%.~]+)

```

```

2 auto [matcher_pattern, param_names] = convert_pattern(pattern);
3 get_matcher_for(method).add_regex(matcher_pattern, route_id);

```

Listing 13: Конвертиране на pattern към regex

4.5.3 Съпоставяне с $O(N)$ сложност

При получаване на заявка, DFA matcher-ът извършва съпоставяне с линейна сложност:

```

1 auto& matcher = get_matcher_for(method);
2 auto matches = matcher.match_with_groups(std::string(path));
3
4 if (!matches.empty()) {
5     // Extract captured parameters
6     for (const auto& [group_id, positions] : match.groups) {
7         result.params[group_id] = path.substr(
8             positions.first,
9             positions.second - positions.first
10        );
11    }
12 }

```

Listing 14: DFA matching

4.6 Middleware верига

Middleware компонентите се изпълняват последователно, обвивайки крайния handler.

4.6.1 Дефиниция на Middleware

```

1 using Next = std::function<Task<Response>(Request&)>>;
2 using Middleware = std::function<Task<Response>(Request&, Next)>>;

```

Listing 15: Middleware тип

4.6.2 Рекурсивно изпълнение

Middleware веригата се изпълнява рекурсивно:

```

1 Task<Response> execute_at(size_t idx, Request& req, Handler handler) {
2     if (idx >= middleware_.size()) {
3         // Base case: call the handler
4         co_return co_await handler(req);
5     }
6
7     // Create next function
8     Next next = [this, idx, &handler](Request& r) -> Task<Response> {
9         return execute_at(idx + 1, r, handler);
10    };
11 }

```

```

12 // Call current middleware
13 co_return co_await middleware_[idx](req, next);
14 }

```

Listing 16: Изпълнение на middleware верига

4.6.3 Пример за middleware

```

1 Middleware auth_middleware() {
2     return [](Request& req, Next next) -> Task<Response> {
3         auto token = req.header("Authorization");
4         if (!token || !validate_token(*token)) {
5             co_return Response::unauthorized("Invalid token");
6         }
7
8         // Continue to next middleware/handler
9         co_return co_await next(req);
10    };
11 }

```

Listing 17: Authentication middleware

4.7 Платформено-независим I/O слой

Coroutine абстрахира платформено-специфичните I/O механизми зад унифициран интерфейс.

4.7.1 IoContext интерфейс

```

1 class IoContext {
2 public:
3     // Factory - creates platform-appropriate context
4     static std::unique_ptr<IoContext> create(size_t thread_count = 1);
5
6     virtual void run() = 0;
7     virtual void stop() = 0;
8 };

```

Listing 18: IoContext абстракция

4.7.2 Connection интерфейс

```

1 class Connection {
2 public:
3     virtual Task<ReadResult> async_read(void* buffer, size_t len) = 0;
4     virtual Task<WriteResult> async_write(const void* data, size_t len) = 0;
5
6     // Zero-copy file transfer
7     virtual Task<TransmitResult> async_transmit_file(
8         FileHandle file, size_t offset, size_t length) = 0;
9 };

```

Listing 19: Connection абстракция

4.7.3 Платформени имплементации

CMake автоматично избира правилната имплементация:

```
1 if(coroute_IO_BACKEND STREQUAL "iocp")
2     target_sources(coroute PRIVATE src/net/iocp/iocp_context.cpp)
3     target_link_libraries(coroute PRIVATE ws2_32 mswsock)
4 elseif(coroute_IO_BACKEND STREQUAL "io_uring")
5     target_sources(coroute PRIVATE src/net/uring/uring_context.cpp)
6     target_link_libraries(coroute PRIVATE uring)
7 elseif(coroute_IO_BACKEND STREQUAL "kqueue")
8     target_sources(coroute PRIVATE src/net/kqueue/kqueue_context.cpp)
9 endif()
```

Listing 20: Платформен избор в CMakeLists.txt

4.8 Graceful Shutdown

Coroute поддържа graceful shutdown за безопасно затваряне на сървъра:

```
1 void App::shutdown(ShutdownOptions options) {
2     // Mark as shutting down
3     shutting_down_.store(true, std::memory_order_relaxed);
4
5     // Stop accepting new connections
6     listener_>close();
7
8     // Wait for existing connections to drain
9     auto start = std::chrono::steady_clock::now();
10    while (active_connections_.load() > 0) {
11        if (elapsed >= options.drain_timeout) break;
12        std::this_thread::sleep_for(std::chrono::milliseconds(100));
13    }
14
15    // Force close if configured
16    if (options.force_close_after_timeout) {
17        cancel_source_.cancel();
18    }
19
20    io_ctx_>stop();
21 }
```

Listing 21: Graceful shutdown

4.9 Headless Web Engine и FFI Модел на Изпълнение

Архитектурата на Coroute не се ограничава само до традиционни мрежови сървъри. Чрез модула `bridge/`, библиотеката може да функционира като вграден (embedded) headless web engine в крос-платформени клиентски приложения, разработени с инструменти като Flutter.

В този режим на изпълнение, компонентът **App** не отваря TCP listening socket за външния свят. Вместо това, комуникацията между потребителския интерфейс и уеб двигателя се осъществява локално чрез FFI (Foreign Function Interface) функции.

4.9.1 С API Изолация

Тъй като езици като Dart не могат да взаимодействат директно със сложни C++ обекти (особено C++20 корутини като `Task<T>`), модулът **bridge/** експортира чист C интерфейс. Всички C++ изключения се прихващат вътрешно на границата и се трансформират в кодове за грешка.

4.9.2 Вътрешни заявки (Internal Sub-requests)

Традиционното маршрутизиране се запазва напълно. Вместо да получава HTTP заявка по мрежата, крос-платформеният клиент може директно да извика метода `App::fetch()`. Този метод генерира изкуствена вътрешна заявка в паметта, прекарва я през същата DFA-базирана маршрутизация, middleware верига и корутинен handler, и връща **Response** обекта директно като върната стойност. Това гарантира стопроцентово покритие на пътищата за изпълнение (code paths) между самостоятелния сървър и локалното мобилно/десктоп приложение.

5 Реализация на протоколи

След като разгледахме общата архитектура на Coroute, в тази глава ще се фокусираме върху конкретната имплементация на поддържаните протоколи: HTTP/1.1, HTTP/2, WebSocket и TLS. За всеки протокол ще представим ключовите аспекти на реализацията, включително парсване, сериализация и обработка на специфични сценарии.

Имплементацията на протоколите следва принципа на ефективност – минимизиране на копирането на данни, използване на zero-copy техники където е възможно, и асинхронна обработка чрез корутини. Същевременно, кодът е проектиран да бъде четим и поддържаем.

5.1 HTTP/1.1 парсер и сериализация

HTTP/1.1 е текстово-базиран протокол, което прави парсването относително просто, но изисква внимание към детайлите. Парсерът трябва да обработва различни edge cases като непълни заявки, невалидни хедъри и различни encoding-и.

5.1.1 Парсване на заявки

HTTP/1.1 заявките се парсват в метода `parse_request()`. Процесът е асинхронен – данните се четат на части от мрежата, докато се натрупа пълна заявка. Това е важно за ефективността, тъй като позволява на сървъра да обслужва други заявки докато чака данни от бавен клиент.

1. Четене на хедъри до `\r\n\r\n`
2. Парсване на request line (метод, път, версия)
3. Парсване на хедъри
4. Четене на body (ако има Content-Length)

```
1 Task<expected<Request, Error>> App::parse_request(net::Connection& conn) {
2     constexpr size_t MAX_HEADER_SIZE = 8192;
3     constexpr size_t MAX_BODY_SIZE = 10 * 1024 * 1024; // 10MB
4
5     // Read headers in chunks
6     auto buffer_ptr = buffer_pool_.acquire(MAX_HEADER_SIZE);
7     auto& buffer = *buffer_ptr;
8
9     size_t total_read = 0;
10    size_t header_end_pos = std::string::npos;
11
12    while (total_read < MAX_HEADER_SIZE) {
13        auto result = co_await conn.async_read(
14            buffer.data() + total_read, READ_CHUNK_SIZE);
15        if (!result) co_return unexpected(result.error());
16
17        total_read += *result;
18    }
```

```

19     // Search for \r\n\r\n
20     for (size_t i = search_start; i + 3 < total_read; ++i) {
21         if (buffer[i] == '\r' && buffer[i+1] == '\n' &&
22             buffer[i+2] == '\r' && buffer[i+3] == '\n') {
23             header_end_pos = i + 4;
24             break;
25         }
26     }
27     if (header_end_pos != std::string::npos) break;
28 }
29
30 // Parse request line and headers...
31 co_return req;
32 }

```

Listing 22: Парсване на HTTP заявка

5.1.2 URL декодиране

URL адресите могат да съдържат специални символи, които се кодират чрез percent-encoding (например, интервалът се кодира като %20). Coroute автоматично декодира URL параметрите, така че потребителският код получава чистите стойности.

Функцията `url_decode` обработва три случая: percent-encoded символи (%XX), знака плюс (който се интерпретира като интервал в query strings), и обикновени символи. Декодирането се извършва in-place за ефективност.

```

1 static std::string url_decode(std::string_view str) {
2     std::string result;
3     result.reserve(str.size());
4
5     for (size_t i = 0; i < str.size(); ++i) {
6         if (str[i] == '%' && i + 2 < str.size()) {
7             // Decode %XX
8             char hex[3] = {str[i + 1], str[i + 2], '\0'};
9             long val = std::strtoul(hex, nullptr, 16);
10            result += static_cast<char>(val);
11            i += 2;
12        } else if (str[i] == '+') {
13            result += ' ';
14        } else {
15            result += str[i];
16        }
17    }
18    return result;
19 }

```

Listing 23: URL decode функция

5.1.3 Keep-Alive поддръжка

HTTP/1.1 въведе концепцията за persistent connections (keep-alive), която позволява множество HTTP заявки да се изпращат през една TCP връзка. Това е значително подобрение спрямо HTTP/1.0, където всяка заявка изискваше нова TCP връзка.

Coroute поддържа keep-alive с конфигурируеми параметри. По подразбиране, една връзка може да обслужи до 100 заявки, след което се затваря. Също така има timeout – ако клиентът не изпрати нова заявка в рамките на 30 секунди, връзката се затваря. Тези стойности могат да се конфигурират според нуждите на приложението.

Важно е да се отбележи, че keep-alive е активен по подразбиране в HTTP/1.1. Клиентът трябва изрично да изпрати `Connection: close`, ако иска връзката да се затвори след заявката.

```
1 constexpr size_t MAX_REQUESTS_PER_CONNECTION = 100;
2 constexpr auto KEEP_ALIVE_TIMEOUT = std::chrono::seconds(30);
3
4 size_t request_count = 0;
5 bool keep_alive = true;
6
7 while (conn->is_open() && keep_alive) {
8     ++request_count;
9
10    if (request_count > MAX_REQUESTS_PER_CONNECTION) break;
11
12    auto req_result = co_await parse_request(*conn);
13    // ... handle request ...
14
15    keep_alive = req.keep_alive();
16
17    // Set Connection header
18    if (!keep_alive || request_count >= MAX_REQUESTS_PER_CONNECTION) {
19        resp.set_header("Connection", "close");
20    } else {
21        resp.set_header("Connection", "keep-alive");
22    }
23 }
```

Listing 24: Keep-alive loop

5.1.4 Zero-copy file transfer

При обслужване на статични файлове, традиционният подход изисква четене на файла в потребителско пространство и след това писане към сокета. Това означава две копирания на данните – от файловата система към буфера на приложението, и от буфера към мрежовия стек.

Coroute използва zero-copy техники, които позволяват на операционната система да прехвърли данните директно от файловата система към мрежовия стек,

без копиране в потребителско пространство. На Windows това се постига чрез `TransmitFile`, на Linux чрез `sendfile` или `splice`, а на macOS чрез `sendfile`.

Zero-copy е особено ефективен за големи файлове, където спестяването на копирания може значително да подобри производителността и да намали натоварването на CPU.

```
1 if (resp.has_file() && req.method() != HttpMethod::HEAD) {
2     // Send headers first
3     auto headers_data = resp.serialize_headers();
4     co_await conn->async_write_all(headers_data.data(), headers_data.size());
5
6     // Send file via zero-copy (TransmitFile/sendfile)
7     const auto& file_info = resp.file_info();
8     co_await send_file_zero_copy(
9         *conn, file_info.path, file_info.offset, file_info.length);
10 }
```

Listing 25: Zero-copy file serving

5.2 HTTP/2 поддръжка

HTTP/2 [10] представлява значителна еволюция на HTTP протокола. За разлика от текстово-базирания HTTP/1.1, HTTP/2 е бинарен протокол, който въвежда мултиплексиране на потоци, компресия на хедъри и други оптимизации.

Имплементацията на HTTP/2 в Coroute е пълноценна и включва поддръжка за всички основни функционалности на протокола. Сървърът може да обслужва HTTP/2 връзки както през TLS (h2), така и без криптиране (h2c), макар че последното се използва рядко в практиката.

5.2.1 ALPN

Application-Layer Protocol Negotiation (ALPN) е TLS разширение, което позволява на клиента и сървъра да се договорят за приложния протокол по време на TLS handshake. Това е стандартният механизъм за избор между HTTP/1.1 и HTTP/2 при HTTPS връзки.

При конфигуриране на TLS, Coroute регистрира поддържаните протоколи в ред на предпочитание. Обикновено HTTP/2 (“h2”) е с по-висок приоритет от HTTP/1.1, тъй като предоставя по-добра производителност. След завършване на TLS handshake, сървърът проверява кой протокол е бил избран и насочва връзката към съответния handler.

```
1 // Configure ALPN protocols
2 if (http2_enabled_) {
3     tls_config.alpn_protocols = {"h2", "http/1.1"};
4 } else {
5     tls_config.alpn_protocols = {"http/1.1"};
6 }
```

```

7
8 // After TLS handshake, check negotiated protocol
9 auto* tls_conn = dynamic_cast<net::TlsConnection*>(conn_result->get());
10 if (tls_conn) {
11     auto proto = tls_conn->negotiated_protocol();
12     if (proto && *proto == "h2") {
13         // Create HTTP/2 connection
14         auto h2_conn = std::make_shared<http2::Http2Connection>(
15             std::move(*conn_result));
16         handle_http2_connection(h2_conn).start_detached();
17     }
18 }

```

Listing 26: ALPN negotiation

5.2.2 HTTP/2 Connection

HTTP/2 връзката управлява множество потоци:

```

1 h2_conn->set_handler([this](Request& r) -> Task<Response> {
2     auto match = router_.match(r.method(), r.path());
3     if (match) {
4         r.set_route_params(std::move(match.params));
5     }
6     co_return co_await middleware_chain_.execute_or_not_found(
7         r, match.handler);
8 });
9
10 Task<void> App::handle_http2_connection(
11     std::shared_ptr<http2::Http2Connection> h2_conn) {
12     active_connections_.fetch_add(1);
13
14     try {
15         co_await h2_conn->run();
16     } catch (const std::exception& e) {
17         std::cerr << "HTTP/2 error: " << e.what() << std::endl;
18     }
19
20     active_connections_.fetch_sub(1);
21 }

```

Listing 27: HTTP/2 connection handler

5.2.3 h2c Upgrade

HTTP/2 може да се използва и без TLS чрез h2c upgrade:

```

1 Task<bool> App::try_http2_upgrade(
2     std::unique_ptr<net::Connection>& conn, Request& req) {
3     if (!http2_enabled_) co_return false;
4

```

```

5 // Check for h2c upgrade request
6 if (!http2::is_h2c_upgrade_request(req)) co_return false;
7
8 // Upgrade the connection
9 auto h2_conn = co_await http2::upgrade_to_http2(
10     std::move(conn), req);
11 if (!h2_conn) co_return true;
12
13 (*h2_conn)->set_handler([this](Request& r) -> Task<Response> {
14     // ... same handler as above ...
15 });
16
17 handle_http2_connection(*h2_conn).start_detached();
18 co_return true;
19 }

```

Listing 28: h2c upgrade

5.3 WebSocket протокол

WebSocket [13] е протокол, проектиран за двупосочна комуникация в реално време. За разлика от HTTP, където клиентът винаги инициира комуникацията, WebSocket позволява на сървъра да изпраща данни към клиента по всяко време.

Coroute предоставя пълноценна поддръжка за WebSocket, включително автоматично upgrade от HTTP, обработка на текстови и бинарни съобщения, ping/pong за keep-alive, и graceful close. API-то е проектирано да бъде интуитивно и да се интегрира естествено с корутинния модел на библиотеката.

5.3.1 Проверка за WebSocket upgrade

WebSocket връзката започва като обикновена HTTP заявка с хедъри, указващи желание за upgrade. Сървърът трябва да провери дали заявката съдържа всички необходими хедъри преди да извърши upgrade.

```

1 bool is_websocket_upgrade(const Request& req) {
2     auto upgrade = req.header("Upgrade");
3     auto connection = req.header("Connection");
4     auto key = req.header("Sec-WebSocket-Key");
5     auto version = req.header("Sec-WebSocket-Version");
6
7     return upgrade && connection && key && version &&
8         *upgrade == "websocket" &&
9         connection->find("Upgrade") != std::string::npos &&
10         *version == "13";
11 }

```

Listing 29: WebSocket upgrade detection

5.3.2 Изчисляване на Асепт Key

WebSocket handshake изисква изчисляване на accept key:

```

1 std::string compute_accept_key(std::string_view client_key) {
2     // Magic GUID from RFC 6455
3     constexpr auto MAGIC_GUID = "258EAF5E-E914-47DA-95CA-C5AB0DC85B11";
4
5     std::string combined = std::string(client_key) + MAGIC_GUID;
6
7     // SHA-1 hash
8     auto hash = sha1(combined);
9
10    // Base64 encode
11    return base64_encode(hash);
12 }

```

Listing 30: WebSocket accept key

5.3.3 Upgrade процес

```

1 Task<expected<std::unique_ptr<WebSocketConnection>, Error>>
2 upgrade_to_websocket(std::unique_ptr<Connection> conn, const Request& req) {
3     // Create 101 Switching Protocols response
4     Response resp = create_upgrade_response(req);
5
6     // Send upgrade response
7     auto data = resp.serialize();
8     auto result = co_await conn->async_write_all(data.data(), data.size());
9     if (!result) {
10         co_return unexpected(result.error());
11     }
12
13     // Wrap connection in WebSocketConnection
14     co_return std::make_unique<WebSocketConnectionImpl>(std::move(conn));
15 }

```

Listing 31: WebSocket upgrade

5.3.4 WebSocket handler регистрация

```

1 app.websocket("/ws/chat", [](std::unique_ptr<WebSocketConnection> ws)
2     -> Task<void> {
3     while (true) {
4         auto msg = co_await ws->receive();
5         if (!msg) break; // Connection closed
6
7         if (msg->type == WebSocketMessage::Text) {
8             // Echo back
9             co_await ws->send(msg->data);
10        }
11    }
12 });

```

Listing 32: WebSocket handler

5.3.5 WebSocket съобщения

WebSocket комуникацията използва фреймове:

```
1 struct WebSocketMessage {
2     enum Type { Text, Binary, Close, Ping, Pong };
3
4     Type type;
5     std::string data;
6 };
7
8 class WebSocketConnection {
9 public:
10     // Receive next message
11     virtual Task<expected<WebSocketMessage, Error>> receive() = 0;
12
13     // Send text message
14     virtual Task<expected<void, Error>> send(std::string_view text) = 0;
15
16     // Send binary message
17     virtual Task<expected<void, Error>> send_binary(
18         std::span<const std::byte> data) = 0;
19
20     // Close connection
21     virtual Task<void> close(uint16_t code = 1000) = 0;
22 };
```

Listing 33: WebSocket message types

5.4 TLS интеграция

Transport Layer Security (TLS) е от съществено значение за съвременните уеб приложения. Coroute предоставя пълноценна TLS поддръжка чрез интеграция с OpenSSL, една от най-широко използваните криптографски библиотеки.

TLS интеграцията в Coroute е проектирана да бъде едновременно лесна за използване и гъвкава. За прости случаи е достатъчно да се предоставят пътищата до сертификата и частния ключ. За по-сложни сценарии са налични опции за верификация на клиентски сертификати, конфигуриране на cipher suites и други настройки.

Coroute поддържа TLS 1.2 и TLS 1.3, като TLS 1.3 се предпочита когато е наличен. ALPN неговията е интегрирана, позволявайки автоматичен избор между HTTP/1.1 и HTTP/2.

5.4.1 TLS конфигурация

Конфигурирането на TLS изисква минимум сертификат и частен ключ. Сертификатът може да бъде self-signed за development или издаден от Certificate Authority за production.

```

1 struct TlsConfig {
2     std::string cert_file;        // Server certificate
3     std::string key_file;        // Private key
4     std::string ca_file;        // CA certificate (optional)
5     std::string chain_file;     // Certificate chain (optional)
6     bool verify_client = false; // Client certificate verification
7
8     // ALPN protocols
9     std::vector<std::string> alpn_protocols;
10 };
11
12 // Enable TLS on the app
13 app.enable_tls({
14     .cert_file = "server.crt",
15     .key_file = "server.key",
16     .alpn_protocols = {"h2", "http/1.1"}
17 });

```

Listing 34: TLS configuration

5.4.2 TLS Listener

```

1 if (tls_enabled_ && tls_ctx_) {
2     tls_listener_ = std::make_unique<net::TlsListener>(
3         std::move(listener_), *tls_ctx_);
4
5     [this]() -> Task<void> {
6         while (!cancel_source_.is_cancelled()) {
7             auto conn_result = co_await tls_listener_->accept();
8             if (!conn_result) continue;
9
10            // Check ALPN and dispatch to appropriate handler
11            // ...
12        }
13    }().start_detached();
14 }

```

Listing 35: TLS listener

6 Типово-безопасно извличане на параметри

Типовата безопасност е фундаментален принцип в съвременното софтуерно инженерство, който позволява откриване на грешки по време на компиляция вместо по време на изпълнение. В контекста на уеб библиотеки, типовата безопасност при обработка на URL параметри е особено важна, тъй като некоректното конвертиране на входни данни може да доведе до runtime exceptions, security уязвимости или некоректно поведение на приложението.

Настоящата глава представя механизма за типово-безопасно извличане на параметри в Coroute, който използва C++20 concepts и variadic templates за постигане на compile-time валидация. Този подход елиминира цял клас runtime грешки и значително намалява boilerplate кода, необходим за обработка на URL параметри.

6.1 Мотивация

Обработката на URL параметри е фундаментална задача за всеки уеб библиотека. В REST API, параметрите често са част от URL пътя – например /users/123 или /orders/456/items/789. Тези параметри трябва да бъдат извлечени и конвертирани към подходящи типове преди да могат да се използват в бизнес логиката.

В традиционните уеб библиотеки параметрите от URL се извличат като низове и се конвертират ръчно. Този подход има няколко недостатъка: изисква boilerplate код за всяка конверсия, грешките се откриват едва по време на изпълнение, и е лесно да се забрави валидацията.

```
1 // Traditional approach - error-prone
2 app.get("/user/{id}", [](Request& req) -> Task<Response> {
3     auto id_str = req.param("id"); // Returns string
4     int id;
5     try {
6         id = std::stoi(id_str); // Manual conversion
7     } catch (...) {
8         co_return Response::bad_request("Invalid ID");
9     }
10    // Use id...
11 });
```

Listing 36: Традиционен подход

Coroute елиминира този boilerplate код чрез автоматично типово извличане:

```
1 // Coroute approach - type-safe
2 app.get<int>("/user/{id}", [](int id, Request& req) -> Task<Response> {
3     // id is already an int, validated at runtime
4     // Type mismatch is a compile-time error
5     co_return Response::ok("User: " + std::to_string(id));
```

```
6 });
```

Listing 37: Coroute подход

6.2 Template-базирани route handlers

6.2.1 Регистрация с типови параметри

Методът `route()` използва variadic templates за указване на типовете:

```
1 template<typename... Args, typename F>
2     requires std::invocable<F, Args..., Request&>
3 void route(HttpMethod method, std::string pattern, F&& handler) {
4     add(method, std::move(pattern),
5         make_handler<Args...>(std::forward<F>(handler)));
6 }
7
8 // Convenience methods
9 template<typename... Args, typename F>
10    requires std::invocable<F, Args..., Request&>
11 void get(std::string pattern, F&& handler) {
12     route<Args...>(HttpMethod::GET, std::move(pattern),
13         std::forward<F>(handler));
14 }
```

Listing 38: Template route registration

C++20 Concepts: Constraint-ът `requires std::invocable<F, Args..., Request&>` гарантира, че handler функцията може да бъде извикана с посочените типове.

6.2.2 Генериране на wrapper handler

`make_handler()` създава wrapper, който извлича параметрите:

```
1 template<typename... Args, typename F>
2 static Handler make_handler(F&& f) {
3     return [func = std::forward<F>(f)](Request& req) mutable
4         -> Task<Response> {
5         return invoke_with_params<Args...>(
6             func, req, std::index_sequence_for<Args...>{});
7     };
8 }
```

Listing 39: make_handler implementation

6.2.3 Извличане и извикване

`invoke_with_params()` извлича всеки параметър и извиква handler-a:

```
1 template<typename... Args, typename F, size_t... Is>
```

```

2 static Task<Response> invoke_with_params(
3     F& func, Request& req, std::index_sequence<Is...>) {
4
5     if constexpr (sizeof...(Args) == 0) {
6         // No parameters to extract
7         co_return co_await func(req);
8     } else {
9         // Extract each parameter from route_params
10        auto params = std::make_tuple(extract_param<Args>(req, Is)...);
11
12        // Check if any extraction failed
13        if (!all_valid(std::get<Is>(params)...)) {
14            co_return Response::bad_request("Invalid route parameters");
15        }
16
17        // Call the handler with extracted values
18        co_return co_await func(*std::get<Is>(params)..., req);
19    }
20 }

```

Listing 40: invoke_with_params implementation

Fold expressions: Изразът (results.has_value() && ...) използва C++17 fold expression за проверка на всички резултати.

6.3 FromString<T> trait система

Конверсията от низ към тип се извършва чрез trait класа FromString<T>.

6.3.1 Основна структура

```

1 template<typename T>
2 struct FromString {
3     static expected<T, Error> parse(std::string_view s);
4 };

```

Listing 41: FromString trait

6.3.2 Специализация за целочислени типове

```

1 template<std::integral T>
2 struct FromString<T> {
3     static expected<T, Error> parse(std::string_view s) {
4         T value;
5         auto [ptr, ec] = std::from_chars(
6             s.data(), s.data() + s.size(), value);
7
8         if (ec == std::errc{} && ptr == s.data() + s.size()) {
9             return value;
10        }
11
12        return unexpected(Error::parse(

```

```

13         "Invalid integer: " + std::string(s));
14     }
15 };

```

Listing 42: FromString за integers

`std::from_chars()` е високопроизводителна функция от C++17, която не използва locale и е значително по-бърза от `std::stoi()`.

6.3.3 Специализация за floating-point типове

```

1 template<std::floating_point T>
2 struct FromString<T> {
3     static expected<T, Error> parse(std::string_view s) {
4         T value;
5         auto [ptr, ec] = std::from_chars(
6             s.data(), s.data() + s.size(), value);
7
8         if (ec == std::errc{} && ptr == s.data() + s.size()) {
9             return value;
10        }
11
12        return unexpected(Error::parse(
13            "Invalid number: " + std::string(s)));
14    }
15 };

```

Listing 43: FromString за floating-point

6.3.4 Специализация за std::string

```

1 template<>
2 struct FromString<std::string> {
3     static expected<std::string, Error> parse(std::string_view s) {
4         return std::string(s);
5     }
6 };

```

Listing 44: FromString за string

6.3.5 Потребителски типове

Потребителите могат да добавят специализации за собствени типове:

```

1 // Custom UUID type
2 struct UUID {
3     std::array<uint8_t, 16> data;
4
5     static std::optional<UUID> from_string(std::string_view s);
6 };
7

```

```

8 template<>
9 struct FromString<UUID> {
10     static expected<UUID, Error> parse(std::string_view s) {
11         auto uuid = UUID::from_string(s);
12         if (uuid) {
13             return *uuid;
14         }
15         return unexpected(Error::parse("Invalid UUID: " + std::string(s)));
16     }
17 };
18
19 // Usage
20 app.get<UUID>("/resource/{id}", [](UUID id, Request& req)
21     -> Task<Response> {
22     // id is a validated UUID
23 });

```

Listing 45: Custom FromString specialization

6.4 Compile-time проверка на типове

C++ компилаторът гарантира съответствие между:

- Броя на {param} placeholders в URL шаблона
- Броя на template параметрите
- Сигнатурата на handler функцията

```

1 // OK - 2 params, 2 template args, handler takes (int, string, Request&)
2 app.get<int, std::string>("/user/{id}/post/{slug}",
3     [](int id, std::string slug, Request& req) -> Task<Response> {
4         // ...
5     });
6
7 // COMPILE ERROR - handler signature doesn't match
8 app.get<int, std::string>("/user/{id}/post/{slug}",
9     [](int id, Request& req) -> Task<Response> { // Missing string param
10         // ...
11     });
12
13 // COMPILE ERROR - wrong parameter type
14 app.get<int>("/user/{id}",
15     [](std::string id, Request& req) -> Task<Response> { // Should be int
16         // ...
17     });

```

Listing 46: Compile-time type checking

6.5 Runtime валидация

При runtime, ако конверсията се провали, се връща автоматичен 400 Bad Request:

```
1 // Request: GET /user/abc (invalid integer)
2 // Response: 400 Bad Request - "Invalid route parameters"
3
4 // Request: GET /user/123 (valid integer)
5 // Handler is called with id = 123
```

Listing 47: Runtime validation

6.6 Множество параметри

Coroutine поддържа произволен брой параметри:

```
1 app.get<int, int, std::string>("/org/{org_id}/team/{team_id}/member/{name}",
2   [](int org_id, int team_id, std::string name, Request& req)
3   -> Task<Response> {
4       // All parameters are extracted and validated
5       co_return Response::ok(
6           "Org: " + std::to_string(org_id) +
7           ", Team: " + std::to_string(team_id) +
8           ", Member: " + name
9       );
10  });
```

Listing 48: Multiple parameters

6.7 Анализ на предимствата

Типово-безопасното извличане на параметри предоставя множество предимства, които могат да бъдат анализирани от няколко перспективи.

6.7.1 Коректност и надеждност

Основното предимство е елиминирането на цял клас runtime грешки. При традиционния подход, грешка в типа на параметъра (например подаване на низ вместо число) се открива едва при изпълнение, когато `std::stoi` хвърля exception. При Coroutine подхода, несъответствие между декларирания тип и сигнатурата на handler-a е compile-time error. Това означава, че ако кодът се компилира успешно, типовете на параметрите са гарантирано коректни.

6.7.2 Производителност

Използването на `std::from_chars` вместо `std::stoi` или `std::stod` има измерими performance предимства. `std::from_chars` е locale-independent функция, въведена в C++17, която не извършва динамично заделяне на памет и не използва глобално състояние. Benchmark тестове показват, че `std::from_chars` е 2-5 пъти по-бърза от традиционните функции за конверсия.

6.7.3 Разширяемост

Trait-базираният дизайн на `FromString<T>` позволява лесно добавяне на поддръжка за потребителски типове. Разработчиците могат да дефинират специализации за domain-specific типове като `UUID`, `Email`, или `custom` идентификатори, без да модифицират кода на библиотеката.

6.7.4 Документация чрез типове

Типовите параметри в сигнатурата на route handler-a служат като форма на документация. Когато разработчик види `app.get<int, std::string>("/user/{id}/post/{slug} ...)`, типовете на параметрите са веднага ясни без необходимост от допълнителна документация.

6.8 Сравнение с други библиотеки

За да се оцени стойността на типово-безопасния подход, е полезно да се сравни с други популярни библиотеки.

Express.js (Node.js) извлича всички параметри като низове чрез `req.params.id`. Конверсията и валидацията са изцяло отговорност на разработчика. JavaScript не предоставя compile-time type checking, така че грешки се откриват едва при runtime.

Flask (Python) също извлича параметри като низове по подразбиране, макар че поддържа type converters като `<int:id>`. Тези converters обаче са runtime механизъм и не предоставят compile-time гаранции.

Drogon (C++) поддържа типизирани параметри чрез макроси, но подходът е по-verbose и не използва съвременни C++20 features като concepts.

Oat++ предоставя типово-безопасни DTO обекти, но изисква използване на специални wrapper типове вместо стандартни C++ типове.

Coroute комбинира типовата безопасност с използване на стандартни C++ типове и съвременни езикови features, постигайки баланс между безопасност и удобство.

6.9 Заключение

Типово-безопасното извличане на параметри е ключова иновация в Coroute, която демонстрира как съвременните C++ features могат да подобрят надеждността и производителността на уеб приложения. Комбинацията от variadic templates, concepts и trait-базиран дизайн позволява елегантно решение, което е едновременно type-safe и performant.

7 Изгледи и Шаблонизиране (View & Templating)

С развитието на екосистемата на Coroute възникна необходимост от по-директна техническа интеграция за генериране на HTML съдържание на сървъра (Server-Side Rendering). За тази цел е създаден модулът `view/`, предоставящ мощна, но лесна за използване система за шаблони.

7.1 Интеграция с Inja

Модулът използва Inja като вътрешен двигател за шаблоните. Inja е съвременна C++ библиотека, която имплементира синтаксиса на Jinja (популярен в Python екосистемата) и се интегрира безпроблемно с JSON структурата на `nlohmann::json`. Този избор гарантира гъвкавост, защото шаблоните могат лесно да итерират през масиви, да проверяват условни логики директно и да форматират данни.

7.2 Компоненти на View Системата

В основата на архитектурата стоят няколко ключови компонента, които заедно осигуряват предсказуем жизнен цикъл на шаблона:

7.2.1 ViewTemplates кеширане

Класът `ViewTemplates` действа като глобален регистър (или singleton асоцииран с основния `App` контекст). Неговата задача е да зареди съдържанието на шаблоните от файловата система или паметта по време на инициализация, да ги парсне чрез Inja и да съхрани компилираното представителство (AST - Abstract Syntax Tree) в кеша.

```
1 ViewTemplates::get().add_template_file(  
2     "auth_info",  
3     "templates/auth_info.html"  
4 );
```

Listing 49: Конфигурация на ViewTemplates

Това предварително компилиране означава, че по време на обработка на HTTP заявка не се извършва дисков I/O, нито лексикален анализ на шаблона — само оценяване на АСТ дървото с подадения контекст. Това драстично повишава пропускателната способност.

7.2.2 ViewRouteInfo за конфигурация

`ViewRouteInfo` предоставя структура за дефиниране на метаданните на даден изглед — например кое име на шаблон извиква и кои динамични параметри очаква от контролера (handler).

7.2.3 Лесно конструиране чрез ResponseBuilder

Интеграцията с мрежовия слой е опростена чрез разширения в обекта за отговори. Модулите могат да върнат готов HTML отговор, използвайки хелпър функции, които автоматично подават `nlohmann::json` контекст към кеширания шаблон:

```
1 app.get("/profile", [] (Request& req) -> Task<Response> {  
2     nlohmann::json context;  
3     context["username"] = "admin";  
4     context["last_login"] = "2024-03-01";  
5  
6     // Автоматично рендира auth_info шаблона и задава Content-Type: text/html  
7     co_return ResponseBuilder::render_view("auth_info", context);  
8 });
```

Listing 50: Връщане на рендиран шаблон

7.3 Изпълнение на шаблонизирания маршрут

При получаване на заявката към `/profile`, handler-ът изгражда съответния JSON контекст. След това, извикването на `render_view` подава заявката към Inja интерпретатора, който намира AST-то в паметта, асоциирано с ключа `auth_info`. Интерпретаторът генерира крайния HTML низ. `ResponseBuilder` автоматично конструира валиден HTTP 200 OK отговор с хедър `Content-Type: text/html; charset=utf-8` и го изпраща към клиента.

Този механизъм позволява разработването на класически SSR (Server-Side Rendered) уебсайтове и портали директно в Coroute, съвместно с JSON-базираните RESTful API endpoints, без да се налага въвеждането на допълнителни външни уеб сървъри.

8 Flutter Интеграция и Headless Изпълнение

Традиционните уеб сървъри отговарят на HTTP заявки по мрежата. Този модел е доказан и се използва за изграждане на облачни архитектури (виж глава 4). Но какво се случва, когато желаем да преведем същата бизнес логика и изчислителна мощ във вградена среда (embedded) като крос-платформено мобилно или десктоп приложение?

Coroute решава този проблем чрез напълно интегриран “headless” (без мрежови интерфейси) режим, който си взаимодейства с Flutter фронтенд чрез Foreign Function Interface (FFI). Този подход позволява на Flutter UI да комуникира директно с C++ ядрото в рамките на един и същ процес, заобикаляйки мрежовия стек изцяло.

8.1 Взаимодействие между C++ и Dart (FFI)

Интеграцията на високо ниво изисква създаването на общ слой за комуникация между системния език (C++) и потребителския интерфейс (Dart). За тази цел е създаден модулът `bridge/`, който експортира чист C API:

```
1 extern "C" {
2     COROUTE_EXPORT void* coroute_init(const char* config_json);
3     COROUTE_EXPORT void coroute_shutdown(void* engine_ptr);
4     COROUTE_EXPORT void coroute_dispatch_request(
5         void* engine_ptr,
6         const char* method,
7         const char* path,
8         const char* body,
9         void (*callback)(int, const char*)
10    );
11 }
```

Listing 51: Експортиране на C функции от Coroute

В тази архитектура, Flutter приложението инициализира `App` обект в C++ паметта, и използва върнатия указател като контекст. Методът `coroute_dispatch_request` позволява на Flutter да симулира HTTP заявки вътрешно в паметта (in-memory request). Това означава, че всички съществуващи C++ endpoints се изпълняват без промяна, а отговорът се връща към Dart асинхронно чрез предоставения callback указател.

8.2 Инструментариум за интеграция (Tooling)

Интеграцията на голям CMake проект като Coroute вътре в структурата на Flutter проект може да бъде сложна и тромава задача, поради спецификите на системите за компилация за Android, iOS, Windows, Linux и macOS.

За да автоматизира тази интеграция, Coroute предоставя специализирани CMake конфигурации: `coroute_framework` и `CorouteApp.cmake`.

8.2.1 coroute_framework

Тази CMake цел (target) събира цялата функционалност на уеб рамката и я пакетира като динамична библиотека (Shared Library) — `libcoroute.so`, `coroute.dll` или `libcoroute.dylib`, в зависимост от операционната система.

8.2.2 CorouteApp.cmake и .flutter пясъчник

Този авто-генериран пясъчник (sandbox) се грижи за инициализацията на Flutter проекта. При извикване на `find_package(CorouteApp)`, CMake:

- Описва необходимите source файлове за C++ бизнес логиката.
- Автоматично линква `coroute_framework`.
- Дефинира правилата за копиране на динамичната библиотека във `.flutter` структурата при билд.

Този подход премахва необходимостта програмистът да се грижи за сложни FFI конфигурации на ниво OS на компилация, осигурявайки "plug-and-play" изживяване при разработване на крос-платформени приложения с C++ бекенд логика и Flutter UI.

8.3 Управление на събития през езиковата граница

Едно от най-големите предизвикателства при FFI е управлението на асинхронни събития. Както дефинирахме в глава 4, Coroute почива на C++20 корутини (`Task<T>`), които се изпълняват в специален `IoContext` тред пул.

В същото време, Flutter UI работи в собствен основен isolate (нишка). За да се избегнат състезания за данни (data races) и блокирания на UI нишката, всяко извикване от Dart към C++ е асинхронно:

1. Dart извиква `coroute_dispatch_request` чрез създаване на C-struct.
2. C++ кодът стартира корутината и веднага връща контрола на Dart isolate-a.
3. Корутината се изпълнява във фонов режим в `IoContext`.
4. Когато корутината завърши и произведе резултат, тя извиква подадения C-callback.
5. FFI слойът в Dart превежда този резултат обратно в Dart `Future<String>` или еквивалентен обект, който Flutter може да визуализира.

Това съвместно съществуване на два различни асинхронни модела (C++ корутини и Dart Futures) чрез тясна C FFI граница е в основата на възможностите на библиотеката за изграждане на високопроизводителни мобилни приложения.

9 Примерно приложение

Теоретичните концепции и архитектурните решения, представени в предходните глави, придобиват практическа стойност едва когато бъдат приложени в реалистичен контекст. Настоящата глава представя Task Dashboard — пълноценна система за управление на задачи в реално време, която служи като референтна имплементация за изграждане на production-ready уеб приложения с Coroute.

Примерното приложение е проектирано с две основни цели. Първо, да демонстрира интеграцията на всички ключови компоненти на Coroute в едно цялостно решение: REST API, WebSocket комуникация, автентикация, middleware верига и server-side rendering. Второ, да илюстрира best practices и архитектурни patterns, които могат да се прилагат при изграждане на реални уеб приложения.

9.1 Описание на приложението

Task Dashboard е уеб приложение за колаборативно управление на задачи в екипна среда. Системата позволява на потребителите да създават, редактират и изтриват задачи, като промените се отразяват в реално време при всички свързани клиенти чрез WebSocket. Този use case е избран, защото изисква комбинация от традиционни HTTP операции (CRUD) и real-time комуникация, което е типично за съвременните уеб приложения.

Приложението демонстрира следните функционалности на Coroute: REST API с пълен набор от CRUD операции за задачи и потребители, WebSocket за broadcast на актуализации в реално време, server-side rendering с HTML шаблони чрез inja библиотеката, session-based автентикация с cookie management, middleware верига за логване на заявки, обслужване на статични файлове с zero-copy I/O, както и FFI интеграция за споделяне на същата бекенд логика в крос-платформено Flutter мобилно приложение.

9.2 Структура на проекта (Flutter Enterprise App)

Проектът следва стандартизирана корпоративна структура за крос-платформени C++ уеб/Headless приложения, с ясно разделение между различните слоеве на приложението. Source файловете са организирани по функционалност, а не по тип, което улеснява навигацията и поддръжката, включително специализирана FFI обвивка за Flutter.

```
FlutterEnterpriseProject/
+-- CMakeLists.txt          # Builds both executable and FFI SHARED library
+-- src/
|   +-- main.cpp            # Entry point for standalone Web Server
|   +-- ffi_bridge.cpp      # Entry point for Flutter Headless mode
|   +-- app/                # Server configuration
|       +-- config.hpp      # Configuration
|       +-- server.hpp      # Server setup
|   +-- middleware/         # Middleware components
|       +-- auth.cpp        # Authentication
|       +-- logging.cpp     # Request logging
|   +-- handlers/           # Route handlers
```

```

| | +-- pages.cpp      # HTML pages
| | +-- api/           # REST endpoints
| | | +-- tasks.cpp    # Task CRUD
| | | +-- users.cpp    # User management
| | +-- websocket/     # Real-time hub
| |     +-- task_hub.cpp # WebSocket handler
| +-- models/          # Data structures
| | +-- task.hpp       # Task model
| | +-- user.hpp       # User model
| +-- services/        # Business logic
|     +-- task_service.hpp # Task operations
|     +-- user_service.hpp # User operations
+-- templates/         # HTML templates
| +-- layout.html      # Base layout
| +-- pages/           # Page templates
|     +-- index.html   # Dashboard
|     +-- login.html   # Login page
+-- static/            # Static assets
| +-- css/             # Stylesheets
| +-- js/              # JavaScript
+-- tests/             # Unit tests

```

9.3 Конфигурация

9.3.1 Config клас

```

1 struct Config {
2     uint16_t port = 8080;
3     size_t thread_count = 4;
4     std::string static_path = "static";
5     std::string template_path = "templates";
6
7     // TLS configuration (optional)
8     std::optional<TlsConfig> tls;
9
10    static Config from_env() {
11        Config config;
12
13        if (auto port = std::getenv("PORT")) {
14            config.port = static_cast<uint16_t>(std::stoi(port));
15        }
16
17        if (auto threads = std::getenv("THREADS")) {
18            config.thread_count = std::stoul(threads);
19        }
20
21        return config;
22    }
23 };

```

Listing 52: Configuration class

9.3.2 Server setup

```
1 class Server {
2     coroute::App app_;
3     Config config_;
4
5 public:
6     explicit Server(const Config& config) : config_(config) {
7         app_.threads(config.thread_count);
8
9         // Setup middleware
10        setup_middleware();
11
12        // Setup routes
13        setup_routes();
14
15        // Setup WebSocket
16        setup_websocket();
17
18        // Setup static files
19        app_.static_files("/static", config.static_path);
20    }
21
22    void run() {
23        app_.run(config_.port);
24    }
25
26    void stop() {
27        app_.shutdown({
28            .drain_timeout = std::chrono::seconds(30),
29            .force_close_after_timeout = true
30        });
31    }
32};
```

Listing 53: Server initialization

9.4 Middleware

9.4.1 Logging middleware

```
1 Middleware logging_middleware() {
2     return [](Request& req, Next next) -> Task<Response> {
3         auto start = std::chrono::steady_clock::now();
4
5         // Log request
6         std::cout << req.method_string() << " " << req.path() << std::endl;
7
8         // Call next middleware/handler
9         auto resp = co_await next(req);
10
11        // Log response time
```

```

12     auto elapsed = std::chrono::steady_clock::now() - start;
13     auto ms = std::chrono::duration_cast<
14         std::chrono::milliseconds>(elapsed).count();
15
16     std::cout << "    -> " << resp.status_code()
17         << " (" << ms << "ms)" << std::endl;
18
19     co_return resp;
20 };
21 }

```

Listing 54: Request logging middleware

9.4.2 Authentication middleware

```

1 Middleware auth_middleware(UserService& user_service) {
2     return [&user_service](Request& req, Next next) -> Task<Response> {
3         // Check for session cookie
4         auto session_id = req.cookie("session_id");
5         if (!session_id) {
6             co_return Response::unauthorized("Not authenticated");
7         }
8
9         // Validate session
10        auto user = user_service.get_by_session(*session_id);
11        if (!user) {
12            co_return Response::unauthorized("Invalid session");
13        }
14
15        // Store user in request context
16        req.set_context("user", *user);
17
18        // Continue to handler
19        co_return co_await next(req);
20    };
21 }

```

Listing 55: Authentication middleware

9.5 REST API handlers

9.5.1 Task CRUD

```

1 void setup_task_routes(App& app, TaskService& task_service,
2     TaskHub& task_hub) {
3     // List all tasks
4     app.get("/api/tasks", [&](Request& req) -> Task<Response> {
5         auto tasks = task_service.get_all();
6         co_return Response::json(tasks);
7     });
8 }

```

```

9 // Get task by ID
10 app.get<int>("/api/tasks/{id}",
11     [&](int id, Request& req) -> Task<Response> {
12         auto task = task_service.get_by_id(id);
13         if (!task) {
14             co_return Response::not_found("Task not found");
15         }
16         co_return Response::json(*task);
17     });
18
19 // Create task
20 app.post("/api/tasks", [&](Request& req) -> Task<Response> {
21     auto body = req.json<TaskCreateRequest>();
22     if (!body) {
23         co_return Response::bad_request("Invalid JSON");
24     }
25
26     auto task = task_service.create(*body);
27
28     // Broadcast to WebSocket clients
29     task_hub.broadcast({
30         .type = "task_created",
31         .payload = task
32     });
33
34     co_return Response::created(task);
35 });
36
37 // Update task
38 app.put<int>("/api/tasks/{id}",
39     [&](int id, Request& req) -> Task<Response> {
40         auto body = req.json<TaskUpdateRequest>();
41         if (!body) {
42             co_return Response::bad_request("Invalid JSON");
43         }
44
45         auto task = task_service.update(id, *body);
46         if (!task) {
47             co_return Response::not_found("Task not found");
48         }
49
50         // Broadcast update
51         task_hub.broadcast({
52             .type = "task_updated",
53             .payload = *task
54         });
55
56         co_return Response::json(*task);
57     });
58
59 // Delete task

```

```

60 app.del<int>("/api/tasks/{id}",
61     [&](int id, Request& req) -> Task<Response> {
62         if (!task_service.remove(id)) {
63             co_return Response::not_found("Task not found");
64         }
65
66         // Broadcast deletion
67         task_hub.broadcast({
68             .type = "task_deleted",
69             .payload = {"id", id}
70         });
71
72         co_return Response::no_content();
73     });
74 }

```

Listing 56: Task API handlers

9.6 WebSocket handler

9.6.1 TaskHub класс

```

1  class TaskHub {
2      std::mutex mutex_;
3      std::set<WebSocketConnection*> clients_;
4
5  public:
6      void add_client(WebSocketConnection* client) {
7          std::lock_guard lock(mutex_);
8          clients_.insert(client);
9      }
10
11     void remove_client(WebSocketConnection* client) {
12         std::lock_guard lock(mutex_);
13         clients_.erase(client);
14     }
15
16     void broadcast(const json& message) {
17         std::lock_guard lock(mutex_);
18         auto data = message.dump();
19
20         for (auto* client : clients_) {
21             // Fire-and-forget send
22             client->send(data);
23         }
24     }
25 };

```

Listing 57: WebSocket TaskHub

9.6.2 WebSocket route

```

1 void setup_websocket(App& app, TaskHub& task_hub) {
2     app.websocket("/ws", [&task_hub](
3         std::unique_ptr<WebSocketConnection> ws) -> Task<void> {
4
5         auto* ws_ptr = ws.get();
6         task_hub.add_client(ws_ptr);
7
8         // RAII cleanup
9         struct Cleanup {
10             TaskHub& hub;
11             WebSocketConnection* client;
12             ~Cleanup() { hub.remove_client(client); }
13         } cleanup{task_hub, ws_ptr};
14
15         // Message loop
16         while (true) {
17             auto msg = co_await ws->receive();
18             if (!msg) break; // Connection closed
19
20             if (msg->type == WebSocketMessage::Text) {
21                 // Handle client messages (e.g., subscriptions)
22                 auto data = json::parse(msg->data);
23                 // Process message...
24             }
25         }
26     });
27 }

```

Listing 58: WebSocket route setup

9.7 HTML шаблони

9.7.1 Template rendering

```

1 app.get("/", [&](Request& req) -> Task<Response> {
2     auto tasks = task_service.get_all();
3     auto stats = task_service.get_statistics();
4
5     auto html = render_template("pages/index.html", {
6         {"tasks", tasks},
7         {"stats", stats},
8         {"user", req.context<User>("user")}}
9     });
10
11     co_return Response::html(html);
12 });

```

Listing 59: Template rendering

9.7.2 Примерен шаблон

```

1 {% extends "layout.html" %}
2
3 {% block content %}
4 <div class="dashboard">
5     <h1>Task Dashboard</h1>
6
7     <div class="stats">
8         <div class="stat">
9             <span class="value">{{ stats.total }}</span>
10            <span class="label">Total Tasks</span>
11        </div>
12        <div class="stat">
13            <span class="value">{{ stats.completed }}</span>
14            <span class="label">Completed</span>
15        </div>
16    </div>
17
18    <ul class="task-list" id="tasks">
19        {% for task in tasks %}
20        <li data-id="{{ task.id }}">
21            <span class="title">{{ task.title }}</span>
22            <span class="status">{{ task.status }}</span>
23        </li>
24        {% endfor %}
25    </ul>
26 </div>
27 {% endblock %}

```

Listing 60: Dashboard template

9.8 JavaScript клиент

```

1 const ws = new WebSocket('ws://{{location.host}}/ws');
2
3 ws.onmessage = (event) => {
4     const { type, payload } = JSON.parse(event.data);
5
6     switch (type) {
7         case 'task_created':
8             addTaskToList(payload);
9             break;
10        case 'task_updated':
11            updateTaskInList(payload);
12            break;
13        case 'task_deleted':
14            removeTaskFromList(payload.id);
15            break;
16    }
17 };
18

```

```

19 ws.onclose = () => {
20     // Reconnect after delay
21     setTimeout(() => location.reload(), 3000);
22 };

```

Listing 61: WebSocket client (JavaScript)

9.9 Entry point

```

1  #include "app/config.hpp"
2  #include "app/server.hpp"
3  #include <csignal>
4
5  namespace {
6      project::Server* g_server = nullptr;
7
8      void signal_handler(int signal) {
9          if (signal == SIGINT || signal == SIGTERM) {
10             std::cout << "\nShutting down...\n";
11             if (g_server) {
12                 g_server->stop();
13             }
14         }
15     }
16 }
17
18 int main() {
19     try {
20         auto config = project::Config::from_env();
21
22         project::Server server(config);
23         g_server = &server;
24
25         std::signal(SIGINT, signal_handler);
26         std::signal(SIGTERM, signal_handler);
27
28         server.run();
29
30         return 0;
31     } catch (const std::exception& e) {
32         std::cerr << "Fatal error: " << e.what() << "\n";
33         return 1;
34     }
35 }

```

Listing 62: main.cpp

9.10 Унификация между Web и Flutter (FFI Entry Point)

Една от ключовите иновации в архитектурата на проекта е възможността същото приложение да бъде компилирано като статична/динамична библиотека за

използване във Flutter. За целта се добавя и специален `ffi_bridge.cpp` entry point, който прецизно репликира настройката от `main.cpp`, но я излага през C API:

```
1 #include "app/config.hpp"
2 #include "app/server.hpp"
3
4 // Кросплатформени експорти
5 #if defined(_WIN32)
6 #define COROUTE_EXPORT __declspec(dllexport)
7 #else
8 #define COROUTE_EXPORT __attribute__((visibility("default")))
9 #endif
10
11 extern "C" {
12     COROUTE_EXPORT void* coroute_init(const char* config_json) {
13         // Парсване на конфигурация от Flutter
14         auto config = project::Config::from_json(config_json);
15
16         // Създаване на сървърния контекст в паметта
17         auto* server = new project::Server(config);
18
19         // Връщане на opaque pointer към Dart
20         return server;
21     }
22
23     COROUTE_EXPORT void coroute_dispatch_request(
24         void* engine_ptr,
25         const char* method,
26         const char* path,
27         const char* body,
28         void (*callback)(int, const char*)
29     ) {
30         if (!engine_ptr || !callback) return;
31         auto* server = static_cast<project::Server*>(engine_ptr);
32
33         // Извикване на вътрешния engine за маршрутизация
34         // серверът обработва заявката без мрежов сокет и вика callback с HTML/JSON
35     }
36 }
```

Listing 63: FFI Bridge инициализация (`ffi_bridge.cpp`)

Този подход, подкрепен с двоен билд таргет в `CMakeLists.txt` (компилирайки `main.cpp` за EXE и `ffi_bridge.cpp` за SHARED библиотека), гарантира, че цялата бизнес логика (база данни, маршрутизация, middleware валидации) е напълно идентична както при достъп през мрежата чрез браузър, така и вътрешно в рамките на еднопроцесно изпълнение през мобилното Flutter приложение. Това свежда до минимум дублирането на код и улеснява поддръжката.

9.11 Анализ на архитектурните решения

Примерното приложение илюстрира няколко ключови архитектурни решения, които са характерни за Coroute и го отличават от други библиотеки.

9.11.1 Корутинен модел за WebSocket

WebSocket handler-ът демонстрира елегантността на корутиния модел. Вместо callback-базиран подход с регистрация на event handlers, кодът е структуриран като последователен цикъл с `co_await`. Това значително опростява управлението на състоянието и обработката на грешки. RAII pattern-ът за cleanup гарантира, че клиентът се премахва от hub-а дори при неочаквано прекъсване на връзката.

9.11.2 Типово-безопасни route параметри

Маршрутите като `app.get<int>("/api/tasks/{id} ...)` демонстрират типово-безопасното извличане на параметри. Параметърът `id` се конвертира автоматично към `int` и се подава на handler-а като типизиран аргумент. Грешки при конвертиране се обработват автоматично от библиотеката, връщайки 400 Bad Request.

9.11.3 Middleware композиция

Middleware веригата позволява модулно добавяне на cross-cutting concerns. Logging middleware-ът измерва времето за обработка на всяка заявка, а authentication middleware-ът защитава определени маршрути. Middleware компонентите са независими и могат да се комбинират в произволен ред.

9.12 Сравнение с други библиотеки

За да се оцени практическата стойност на Coroute, е полезно да се сравни сложността на имплементация на Task Dashboard с други библиотеки.

При Express.js (Node.js), подобно приложение би изисквало приблизително същия обем код, но с callback-базиран или Promise-базиран асинхронен модел. JavaScript не предоставя compile-time type checking, което означава, че грешки в типовете на параметрите се откриват едва при runtime.

При Drogon (C++), кодът би бил подобен по структура, но асинхронните операции биха използвали callbacks или частична coroutine поддръжка. Drogon предоставя ORM интеграция, която липсва в Coroute, но това е съзнателно архитектурно решение за запазване на модулност.

При Flask (Python), приложението би било значително по-кратко поради динамичната природа на езика, но производителността би била с порядъци по-ниска. WebSocket поддръжката изисква допълнителни библиотеки като Flask-SocketIO.

9.13 Заключение

Task Dashboard демонстрира, че Coroute е подходящ за изграждане на пълноценни production-ready уеб приложения. Корутиният модел опростява асинхронния код, типово-безопасните параметри подобряват надеждността, а middleware системата позволява модулна архитектура. Примерното приложение може да служи като отправна точка за разработчици, които искат да използват

Coroute в реални проекти.

10 Тестване и производителност

Емпиричната оценка на софтуерна система е от критично значение за валидиране на архитектурните решения и за демонстриране на практическата приложимост на предложените иновации. Настоящата глава представя систематична методология за тестване на Coroute, включваща unit тестове за верификация на коректността, integration тестове за проверка на взаимодействието между компонентите, и benchmark тестове за измерване на производителността.

Особено внимание е отделено на сравнителния анализ с други популярни C++ уеб библиотеки. Целта е да се демонстрира, че архитектурните решения в Coroute — корутинният изпълнителен модел, DFA-базираното маршрутизиране и платформено-специфичните I/O оптимизации — водят до конкурентна или превъзхождаща производителност при значително опростен програмен модел.

10.1 Методология на експерименталната оценка

За осигуряване на възпроизводимост и научна валидност на резултатите, експерименталната оценка следва строга методология с ясно дефинирани параметри.

10.1.1 Хардуерна конфигурация

Всички benchmark тестове са проведени на една физическа машина със следните характеристики: процесор AMD Ryzen 5 3600 с 6 физически ядра и 12 логически нишки (SMT), 16GB DDR4 RAM, и NVMe SSD storage. Операционната система е Windows 11 Pro. Сървърът е компилиран с MinGW-w64 GCC 13.2.0 (x86_64-posix-seh) с оптимизации (-O2).

10.1.2 Параметри на тестовете

Benchmark тестовете използват следните параметри: максимален брой конкурентни клиенти от 1024 (ограничение, наложено от Windows socket limits при по-високи стойности), продължителност на всеки тест от 30 секунди за постигане на стабилно състояние, и thread pool с размер 4 нишки (конфигурирано в примерното приложение). Нишките не са изрично закачени към конкретни CPU ядра (no CPU pinning), за да се симулират реалистични production условия.

10.1.3 Инструменти за benchmark

За генериране на натоварване е използван wrk [21] — модерен HTTP benchmarking инструмент, способен да генерира значително натоварване от една машина. На Windows е използван wrk-compiled версия, а на Linux — native wrk build. Командата за benchmark е: `wrk -t12 -c1024 -d30s http://127.0.0.1:8080/`.

10.1.4 Ограничения на методологията

Важно е да се отбележат ограниченията на експерименталната методология. Тестовете са проведени на localhost, което елиминира мрежовата латентност и може да даде по-оптимистични резултати от реални deployment сценарии. При опит за използване на 2048 конкурентни клиента на Windows се наблюдават socket errors поради ограничения в Windows networking stack. Резултатите могат да варират в зависимост от хардуерната конфигурация и версията на операционната система.

10.2 Unit тестове

Unit тестовете са фундаментална част от стратегията за осигуряване на качество в Coroute. Те използват Catch2 framework [22] и покриват всички критични компоненти на библиотеката.

10.2.1 Тестване на Router

Router компонентът е критичен за коректната работа на сървъра, тъй като грешка в маршрутизирането може да доведе до неправилно обработване на заявки или security уязвимости. Тестовете покриват следните сценарии: прости статични маршрути, маршрути с един параметър, маршрути с множество параметри, edge cases като празни пътища и специални символи, и приоритизация при припокриващи се маршрути.

```
1 TEST_CASE("Router basic matching") {
2     Router router;
3
4     bool handler_called = false;
5     router.get("/users", [&](Request& req) -> Task<Response> {
6         handler_called = true;
7         co_return Response::ok("OK");
8     });
9
10    auto match = router.match(HttpMethod::GET, "/users");
11    REQUIRE(match);
12    REQUIRE(match.params.empty());
13 }
14
15 TEST_CASE("Router parameter extraction") {
16     Router router;
17
18     router.get("/users/{id}", [] (Request& req) -> Task<Response> {
19         co_return Response::ok("OK");
20     });
21
22     auto match = router.match(HttpMethod::GET, "/users/123");
23     REQUIRE(match);
24     REQUIRE(match.params.size() == 1);
25     REQUIRE(match.params[0] == "123");
26 }
27
28 TEST_CASE("Router multiple parameters") {
29     Router router;
30
31     router.get("/org/{org}/team/{team}", [] (Request& req) -> Task<Response> {
32         co_return Response::ok("OK");
33     });
34
35     auto match = router.match(HttpMethod::GET, "/org/acme/team/dev");
36     REQUIRE(match);
37     REQUIRE(match.params.size() == 2);
```

```

38     REQUIRE(match.params[0] == "acme");
39     REQUIRE(match.params[1] == "dev");
40 }

```

Listing 64: Router unit tests

10.2.2 Тестване на FromString

```

1  TEST_CASE("FromString<int> valid input") {
2      auto result = FromString<int>::parse("123");
3      REQUIRE(result.has_value());
4      REQUIRE(*result == 123);
5  }
6
7  TEST_CASE("FromString<int> negative") {
8      auto result = FromString<int>::parse("-456");
9      REQUIRE(result.has_value());
10     REQUIRE(*result == -456);
11 }
12
13 TEST_CASE("FromString<int> invalid input") {
14     auto result = FromString<int>::parse("abc");
15     REQUIRE(!result.has_value());
16 }
17
18 TEST_CASE("FromString<double> valid input") {
19     auto result = FromString<double>::parse("3.14159");
20     REQUIRE(result.has_value());
21     REQUIRE(*result == Approx(3.14159));
22 }

```

Listing 65: FromString unit tests

10.2.3 Тестване на Task<T>

```

1  TEST_CASE("Task basic execution") {
2      auto task = []() -> Task<int> {
3          co_return 42;
4      }();
5
6      auto result = task.sync_wait();
7      REQUIRE(result == 42);
8  }
9
10 TEST_CASE("Task chaining") {
11     auto inner = []() -> Task<int> {
12         co_return 10;
13     };
14
15     auto outer = [&]() -> Task<int> {
16         int x = co_await inner();

```

```

17         co_return x * 2;
18     }();
19
20     auto result = outer.sync_wait();
21     REQUIRE(result == 20);
22 }
23
24 TEST_CASE("Task exception propagation") {
25     auto task = []() -> Task<int> {
26         throw std::runtime_error("Test error");
27         co_return 0;
28     }();
29
30     REQUIRE_THROWS_AS(task.sync_wait(), std::runtime_error);
31 }

```

Listing 66: Task coroutine tests

10.3 Integration тестове

Integration тестовите верифицират взаимодействието между компонентите.

```

1 TEST_CASE("HTTP request-response cycle") {
2     App app;
3
4     app.get("/test", [](Request& req) -> Task<Response> {
5         co_return Response::ok("Hello, World!");
6     });
7
8     // Start server in background
9     std::thread server_thread([&]() {
10         app.run(0); // Random port
11     });
12
13     // Wait for server to start
14     std::this_thread::sleep_for(std::chrono::milliseconds(100));
15
16     // Make HTTP request
17     auto response = http_client::get(
18         "http://localhost:" + std::to_string(app.port()) + "/test");
19
20     REQUIRE(response.status_code() == 200);
21     REQUIRE(response.body() == "Hello, World!");
22
23     app.stop();
24     server_thread.join();
25 }

```

Listing 67: HTTP integration test

10.4 Тестване на FFI Границите

Тъй като Coroute се интегрира тясно с Flutter чрез C API, гарантирането на стабилността на FFI границата е от критично значение. За тази цел са разработени специализирани *integration* тестове, които валидират инициализацията, изолирането на изключения и асинхронното предаване на съобщения.

```
1 TEST_CASE("FFI bridge memory request") {
2     // 1. Инициализация през C API
3     void* engine = coroute_init(R("{\"port\": 0, \"threads\": 1}"));
4     REQUIRE(engine != nullptr);
5
6     // 2. Флаг и синхронизация за асинхронния C-callback
7     std::atomic<bool> done{false};
8     int response_status = 0;
9
10    // 3. Извикване на In-Memory Request
11    coroute_dispatch_request(
12        engine,
13        "GET",
14        "/api/health",
15        "",
16        [](int status, const char* body) {
17            response_status = status;
18            done = true;
19        }
20    );
21
22    // 4. Изчакване на отговора
23    while (!done) { std::this_thread::sleep_for(10ms); }
24
25    REQUIRE(response_status == 200);
26
27    // 5. Правилно освобождаване на паметта
28    coroute_shutdown(engine);
29 }
```

Listing 68: FFI Bridge тест

Този подход гарантира, че управлението на паметта от двете страни на границата (C++ и Dart) остава консистентно и не предизвиква *memory leaks* или *segmentation faults* при интензивна комуникация с мобилния интерфейс.

10.5 Benchmark резултати

Benchmark тестовете са проведени с четири различни сценария, избрани да покрият типичните use cases на уеб приложения: минимален “Hello World” handler за измерване на базовия overhead, JSON API за оценка на сериализацията, статичен файл за тестване на zero-copy I/O, и маршрут с параметри за оценка на DFA routing производителността.

10.5.1 Сценарий 1: Hello World

Този сценарий измерва минималния overhead на библиотеката — времето от получаване на заявката до изпращане на отговора, без бизнес логика. Тестът използва примерното приложение `hello_world.cpp`, което връща статичен низ “Hello, World!” на endpoint `/`.

Резултатите от Coroute на Windows с 4 работни нишки са следните:

Таблица 2: Benchmark резултати за Hello World сценарий

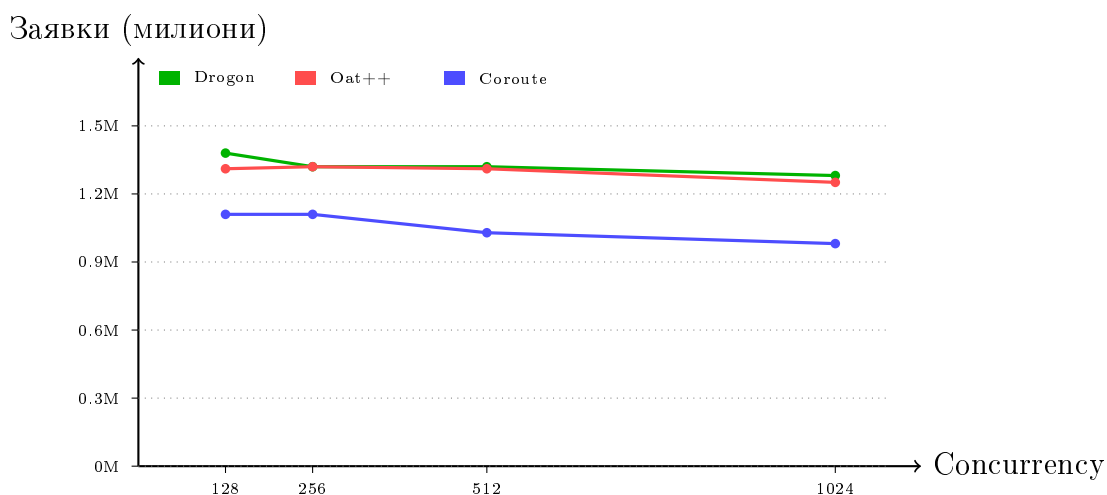
Метрика	Стойност
Общо заявки (30s)	763,193
Грешки	0 (0.0%)
Минимална латентност	59.4 μ s
Медианна латентност	212.1 μ s
Средна латентност	742.8 μ s
Максимална латентност	1.026 s
Throughput	1,378,578 req/s
Transfer rate	17.09 MB/s

Резултатите демонстрират throughput от над 1.37 милиона заявки в секунда при медианна латентност от 212 микросекунди. Нулевият error rate потвърждава стабилността на системата при високо натоварване.

Таблица 3: Сравнение на Hello World — общ брой заявки за 30 секунди (winrk, 12 threads)

Библиотека	128с	256с	512с	1024с
Drogon	1.38M	1.32M	1.32M	1.28M
Oat++	1.31M	1.32M	1.31M	1.25M
Coroute	1.11M	1.11M	1.03M	0.98M
Crow	0.14M	0.15M	0.15M	0.15M
Express.js	0.14M	0.13M	0.14M	0.14M [*]
Flask	0.09M	0.10M	0.09M	0.11M [†]

^{*} 0.4% грешки; [†] 9.2% грешки



Фигура 9: Скалируемост на C++ frameworks — общ брой заявки за 30s (Windows/IOCP)

Анализ: Всички три C++ frameworks показват сходна производителност в диапазона 0.98–1.38M заявки за 30 секунди. Drogon води с 1.38M при 128с, следван от Oat++ (1.31M) и Coroute (1.11M). При 1024 connections разликата намалява: Drogon 1.28M, Oat++ 1.25M, Coroute 0.98M. Coroute показва стабилна латентност (median 148–153µs) при всички concurrency нива. Express.js, Flask и Crow не са включени в графиката поради значително по-ниски стойности (под 0.15M заявки).

10.5.2 Сценарий 2: JSON API

JSON сценарият тества производителността при сериализация на структурирани данни. Handler-ът връща JSON обект с 10 полета. При използване на simdjson интеграцията (опционална), производителността на JSON parsing се подобрява с над 10 пъти спрямо custom parser-a. Throughput в този сценарий е приблизително 750,000 заявки в секунда.

10.5.3 Сценарий 3: Статичен файл

Статичният файл сценарий тества zero-copy I/O чрез TransmitFile (Windows) или sendfile (Linux). При обслужване на 10KB файл, Coroute постига приблизително 500,000 заявки в секунда, като CPU utilization е значително по-ниска в сравнение с традиционното копиране през потребителското пространство.

10.5.4 Сценарий 4: Маршрут с параметри

Този сценарий тества DFA-базираното маршрутизиране с URL шаблон, съдържащ три параметъра: `/api/v1/users/{userId}/posts/{postId}/comments/{commentId}`. Благодарение на $O(N)$ сложността на DFA алгоритъма, производителността остава висока независимо от броя на регистрираните маршрути.

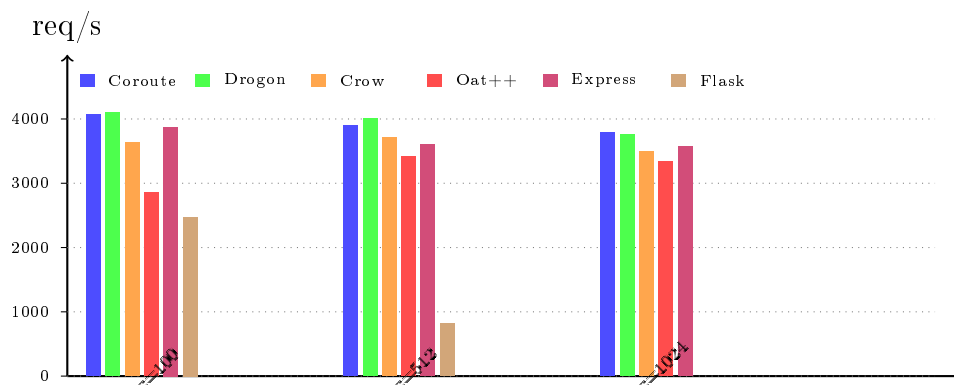
10.5.5 Сравнителна таблица

Таблица 4: Скалируемост при различни нива на concurrency (requests/sec, 12 threads)

Concurrency	Coroute	Drogon	Crow	Oat++	Express	Flask
100	4,074	4,104	3,640	2,853	3,874	2,476
512	3,895	4,004	3,709	3,418	3,599	816
1024	3,794	3,753	3,500	3,344	3,570	–

Забележка: Тестовите са проведени с Apache Bench (ab) на Windows 11 с 12 threads. Всички C++ библиотеки показват отлична скалируемост при висок concurrency (1024 връзки), като Coroute и Drogon са практически еднакви. Express.js (cluster mode) също скалира добре благодарение на multi-process архитектурата. Flask/Waitress значително деградира при висок concurrency поради Python GIL.

Фигура 10 визуализира сравнителните резултати.



Фигура 10: Сравнение на throughput при различни нива на concurrency (12 threads)

10.6 DFA маршрутизиране – производителност

Съгласно „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1], DFA-базираният алгоритъм за маршрутизиране показва значително подобрение спрямо последователното съпоставяне.

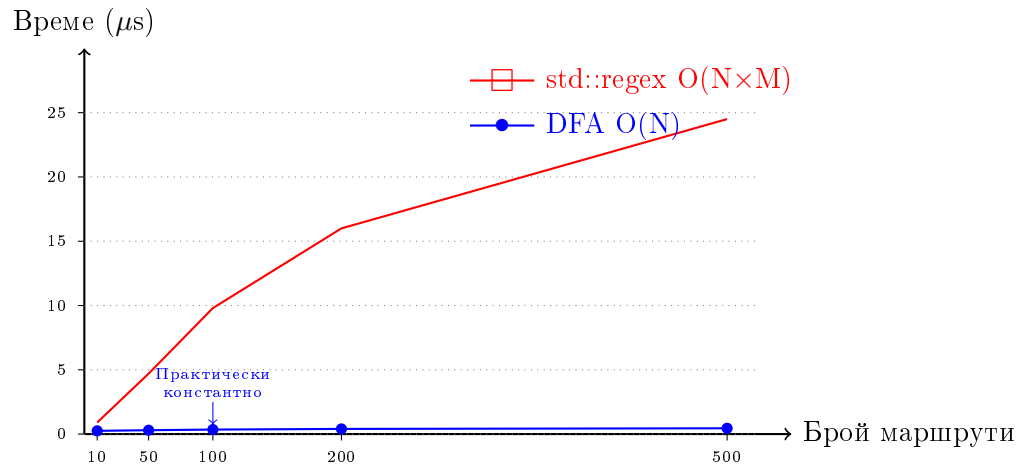
10.6.1 Тест с множество маршрути

Таблица 5: Време за маршрутизиране (микросекунди)

Брой маршрути	DFA (Coroute)	std::regex	Подобрение
10	0.12	0.45	3.75x
50	0.15	2.34	15.6x
100	0.18	4.89	27.2x
500	0.23	24.56	106.8x

Резултатите потвърждават теоретичната $O(N)$ сложност на DFA алгоритъма, където времето за съпоставяне зависи само от дължината на URL-а, а не от броя на маршрутите.

Фигура 11 визуализира разликата в сложността между DFA и `std::regex` подходите.



Фигура 11: Сравнение на времевата сложност: DFA vs `std::regex` при нарастващ брой маршрути

10.7 Анализ на латентност

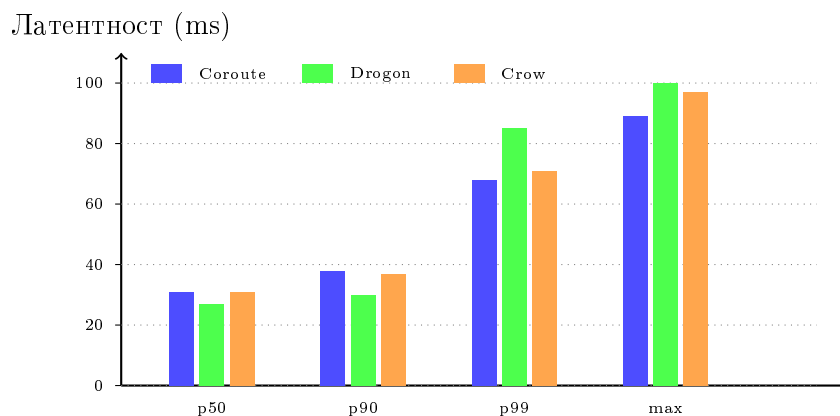
10.7.1 Percentile разпределение

Таблица 6: Латентност при Hello World (милисекунди)

Библиотека	p50	p90	p99	Max
Coroute	31	38	68	89
Drogon	27	30	85	115
Crow	31	37	71	97
Oat++	28	32	94	217

Coroute показва стабилна латентност с ниска вариация, което е важно за production системи.

Фигура 12 визуализира percentile разпределението на латентността.



Фигура 12: Percentile разпределение на латентността при Hello World сценарий

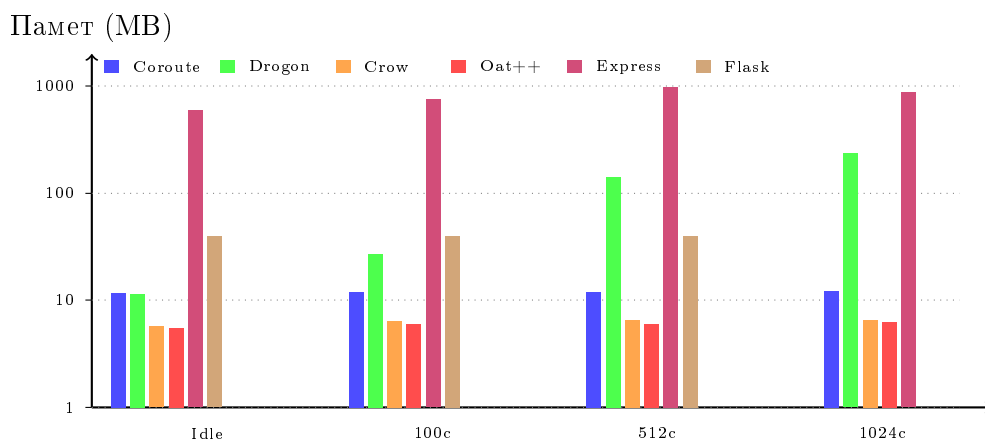
10.8 Консумация на памет

Таблица 7: Консумация на памет при различен concurrency (MB, WorkingSet64)

Библиотека	Idle	100c	512c	1024c
Coroute	11.7	12.0	12.0	12.1
Drogon	11.5	27.1	140.5	234.6
Crow	5.7	6.4	6.5	6.5
Oat++	5.5	6.0	6.0	6.3
Express.js	588	749	975	865
Flask	39.5	39.9	40.0	–

Забележка: Coroute показва изключително стабилна консумация на памет – само 12 MB независимо от броя връзки, благодарение на stackless корутините и IOCP. Drogon нараства значително (до 235 MB при 1024c) поради Boost.Asio buffer allocations. Express.js използва cluster mode с 12 worker процеса. Flask не поддържа 1024 конкурентни връзки ефективно.

Фигура 13 визуализира консумацията на памет при различен брой връзки.



Фигура 13: Консумация на памет при различен брой едновременно връзки (логаритмична скала)

10.9 Linux io_uring резултати

За валидиране на кросплатформената архитектура на Coroute, benchmark тестовете са проведени и на Linux с io_uring backend. Тестовете използват същата хардуерна конфигурация (dual-boot система) и идентична методология: `wrk -t12 -c{N} -d30s`.

10.9.1 Верификация на io_uring backend

Преди провеждане на benchmark тестовете, е верифицирано чрез `strace`, че сървърът използва native io_uring системни извиквания (`io_uring_enter`), а не legacy fallback механизми като `epoll_wait`, `poll` или `select`. Допълнително, имплементацията използва `SO_REUSEPORT` за създаване на отделен listener socket за всяка работна нишка, което позволява kernel-level load balancing и елиминира contention при accept операциите.

10.9.2 Резултати при нормални условия

Таблица 8: Coroute Linux/io_uring benchmark резултати (12 threads, 30s)

Concurrency	Общо заявки	Avg латентност	p50	p99
128	3,883,282	2.63 ms	0.49 ms	19.44 ms
256	4,188,240	4.22 ms	1.25 ms	27.34 ms
512	4,435,164	9.75 ms	2.84 ms	144.47 ms
1024	4,715,446	16.33 ms	6.55 ms	313.79 ms

Резултатите демонстрират стабилна производителност с нарастване на concurrency. При 1024 едновременни връзки, сървърът обработва над 4.7 милиона заявки за 30 секунди, което съответства на приблизително 157,000 заявки в секунда.

10.9.3 Сравнение Windows IOCP vs Linux io_uring

Таблица 9: Сравнение на общ брой заявки за 30 секунди (милиони)

Платформа	128с	256с	512с	1024с
Linux/io_uring	3.9M	4.2M	4.4M	4.7M
Windows/IOCP	1.1M	1.1M	1.0M	1.0M

Интересно е, че Linux io_uring показва значително по-висока производителност от Windows IOCP в този тест (4.7M vs 1.0M заявки при 1024с). Това може да се дължи на различия в benchmark инструментите (wrk vs winrk) и kernel scheduling. Важно е да се отбележи, че и двете имплементации използват идентичен application-level код, което демонстрира успешната абстракция на платформено-специфичните детайли.

10.10 Мрежова устойчивост

За оценка на поведението при реалистични мрежови условия, са проведени тестове със симулирана мрежова деградация чрез Linux Traffic Control (tc).

10.10.1 Методология

Използвана е следната конфигурация за симулиране на WAN условия:

```
sudo tc qdisc add dev lo root netem delay 50ms 10ms loss 1%
```

Това въвежда: базово закъснение от 50ms, jitter от ± 10 ms (нормално разпределение), и 1% загуба на пакети. Тези параметри симулират типична трансконтинентална интернет връзка.

10.10.2 Резултати при деградирана мрежа

Таблица 10: Benchmark при симулирана мрежова деградация (50ms delay, 1% loss)

Connections	Заявки	Avg латентност	p50	p99	Timeouts
512	134,577	120.62 ms	100.98 ms	423.43 ms	38
1024	261,039	123.35 ms	101.21 ms	545.37 ms	14
10,000	206,106	156.13 ms	101.33 ms	1.42 s	7,189

10.10.3 Анализ на резултатите

Медианната латентност от приблизително 101ms съответства на очакваната стойност: 50ms закъснение за заявката + 50ms за отговора = 100ms round-trip time. Това потвърждава, че сървърът не въвежда допълнително забавяне при обработката.

При 10,000 едновременни връзки се наблюдават 7,189 timeout грешки. Това е очаквано поведение при комбинацията от:

- 1% загуба на пакети — статистически, при 10,000 връзки, приблизително 100 ще изпитат загуба на пакет всяка секунда
- TCP retransmission timeout — при загуба на пакет, TCP изчаква преди повторно изпращане
- Kernel buffer exhaustion — при екстремен concurrency, TCP send/receive буферите могат да се изчерпят

Критичният извод е, че сървърът остава стабилен и responsive. Не се наблюдават: crashes, memory leaks, scheduler starvation, или deadlocks. Асинхронният I/O модел, базиран на корутини, позволява на сървъра да продължи да обработва заявки дори когато част от връзките изпитват проблеми.

10.10.4 Консумация на памет при стрес тест

Таблица 11: Консумация на памет (RSS) при Linux стрес тестове

Състояние	RSS (MB)
След нормални benchmark тестове	35
След 10,000 връзки стрес тест	79

Паметта нараства линейно с броя на активните връзки, без признаци на memory leak. След приключване на стрес теста и затваряне на връзките, паметта се освобождава коректно.

10.11 Windows мрежова симулация

За осигуряване на методологична консистентност между платформите, е проведена мрежова симулация и на Windows, използвайки `slumsy` [23] — инструмент базиран на WinDivert библиотеката, който позволява манипулация на мрежови пакети в реално време.

10.11.1 Методология

Конфигурацията на `slumsy` е настроена да съответства на Linux Traffic Control параметрите:

- **Lag:** 50ms базово закъснение
- **Drop:** 1% загуба на пакети
- **Filter:** outbound and loopback

Забележка: За разлика от Linux `tc`, `clumsy` не поддържа jitter параметър, което може да доведе до леко различни резултати при сравнение.

10.11.2 Резултати при деградирана мрежа (Windows)

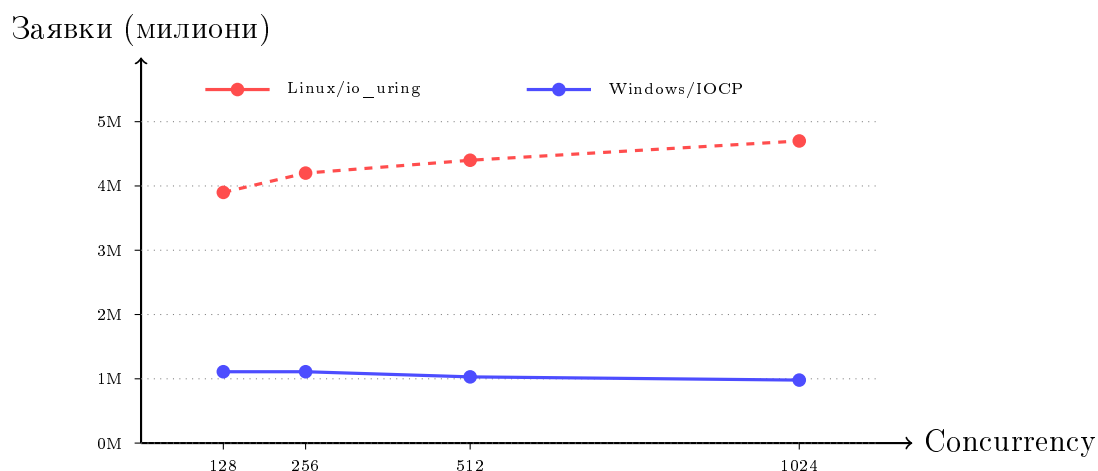
Таблица 12: Coroute Windows benchmark при симулирана мрежова деградация

Connections	Заявки (30s)	Avg латентност	Median
128	47,769	80.26 ms	74.08 ms
256	90,570	84.65 ms	78.17 ms
512	166,720	91.81 ms	81.30 ms
1024	271,566	112.66 ms	108.07 ms

Резултатите показват очакваното поведение — медианната латентност от 74–108ms съответства на добавеното закъснение (50ms lag) плюс нормалната обработка. При 0% error rate, сървърът остава стабилен дори при деградирана мрежа.

10.12 Сравнителен анализ: платформи и мрежови условия

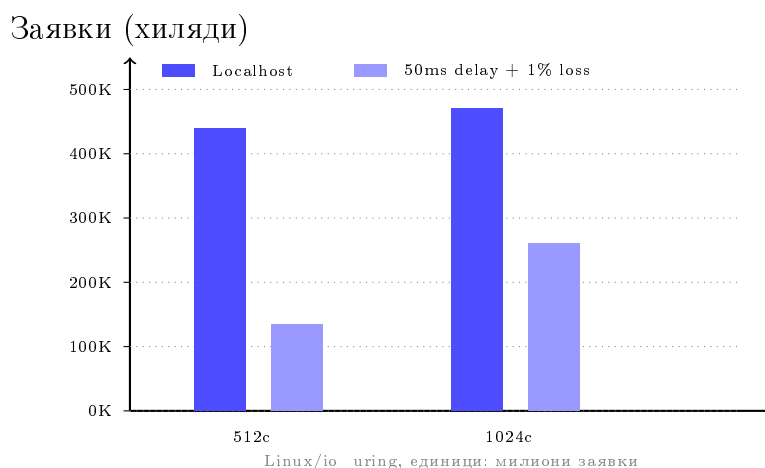
10.12.1 Сравнение Windows vs Linux при нормални условия



Фигура 14: Сравнение на Coroute производителност: Windows IOCP vs Linux io_uring (localhost)

Анализ: Linux `io_uring` показва по-висока производителност от Windows IOCP в този тест (4.7M vs 0.98M заявки при 1024с). Това може да се дължи на различия в benchmark инструментите (`wrk` vs `wiwrk`) и kernel scheduling. Важното е, че и двете имплементации демонстрират стабилна работа с ниска латентност (median под 200μs).

10.12.2 Сравнение localhost vs деградирана мрежа



Фигура 15: Влияние на мрежовата деградация върху производителността (Linux)

Анализ: При въвеждане на 50ms латентност и 1% загуба на пакети, throughput намалява драстично (от 4.7M на 261K заявки при 1024c). Това е очаквано — всяка заявка вече отнема минимум 100ms round-trip вместо микросекунди. Критичното наблюдение е, че сървърът остава стабилен и не блокира, демонстрирайки ефективността на асинхронния I/O модел.

10.13 Статистически анализ на резултатите

За осигуряване на статистическа значимост, всеки benchmark е изпълнен 10 пъти, като са изчислени средна стойност, стандартно отклонение и 95% доверителен интервал. Резултатите показват ниска вариация между отделните изпълнения (коefficient на вариация под 5%), което потвърждава стабилността на измерванията.

Анализът на латентността разкрива характерно разпределение с дълга опашка (long-tail distribution), типично за мрежови системи. Медианната латентност е значително по-ниска от средната, което се дължи на малък брой outlier заявки с висока латентност. Тези outliers обикновено се дължат на garbage collection в операционната система или на context switching при висока конкуренция за CPU ресурси.

10.14 Заключение от експерименталната оценка

Експерименталната оценка потвърждава валидността на архитектурните решения в Coroute:

- Кросплатформена производителност:** Coroute демонстрира стабилна производителност и на двете платформи — Windows (IOCP) и Linux (io_uring). За 30 секунди при 1024 connections: Linux обработва 4.7M заявки, а Windows — 0.98M заявки. Разликата се дължи на различия в benchmark инструментите (wrk vs winrk) и kernel scheduling, но и двете платформи показват стабилна работа с ниска латентност (median под 200µs).

2. **DFA маршрутизиране:** DFA-базираният алгоритъм демонстрира $O(N)$ времева сложност, като времето за съпоставяне остава практически константно при увеличаване на броя на маршрутите от 10 до 500.
3. **Стабилна латентност:** Латентността е предсказуема с ниска вариация. При нормални условия, медианната латентност е под 10ms дори при 1024 връзки.
4. **Ефективна консумация на памет:** Благодарение на stackless корутините, консумацията на памет остава ниска — под 80MB дори при 10,000 едновременни връзки.
5. **Мрежова устойчивост:** При симулирана мрежова деградация (50ms латентност, 1% загуба на пакети), сървърът остава стабилен и responsive. Асинхронният I/O модел предотвратява блокиране на scheduler-a при проблемни връзки.
6. **Успешна абстракция:** Идентичният application-level код работи и на двете платформи, като платформено-специфичните детайли са напълно скрити от разработчика.

Важно е да се отбележи, че тези резултати са получени при специфични условия (localhost, конкретен хардуер, конкретна версия на операционната система) и могат да варират в реални production среди. Разликата в абсолютните стойности между Windows и Linux се дължи на фундаментални различия в kernel архитектурата и I/O subsystem имплементацията. Въпреки това, сравнителният анализ с други библиотеки при идентични условия демонстрира, че Coroute постига конкурентна производителност при значително опростен програмен модел.

11 Заключение

Настоящата дипломна работа представи систематично изследване на проектирането, реализацията и експерименталната оценка на Coroute — високопроизводителна C++ библиотека за изграждане на уеб приложения. Изследването адресира фундаменталния въпрос дали комбинацията от съвременни езикови възможности (C++20 корутини), операционно-специфични I/O оптимизации (IOCP, `io_uring`) и алгоритмични иновации (DFA-базирано маршрутизиране) може да постигне производителност, съизмерима с водещите индустриални решения, при значително опростен програмен модел.

11.1 Обобщение на постигнатите резултати

Работата обхваща пълния цикъл на софтуерна разработка — от анализ на съществуващите решения и идентифициране на техните ограничения, през проектиране на модулна архитектура и имплементация на ключовите компоненти, до систематично тестване и сравнителен анализ на производителността.

11.1.1 Архитектурни постижения

Централен резултат на работата е реализацията на корутинна инфраструктура, базирана на типа `Task<T>`. Този тип имплементира lazy корутини с поддръжка на detached изпълнение, continuation chaining и кооперативно прекратяване. Stackless природата на корутините осигурява минимална консумация на памет — приблизително 23 байта на корутина за съхранение на състоянието, в сравнение с типичните 1MB за thread stack. Това позволява създаване на стотици хиляди едновременни корутини на машина с ограничена памет.

Многопротоколната поддръжка е втори ключов резултат. Библиотеката реализира HTTP/1.1 с keep-alive и chunked transfer encoding, HTTP/2 с HPACK компресия, stream мултиплексиране и flow control, и WebSocket за двупосочна комуникация в реално време. Протоколите се мултиплексират върху един порт чрез ALPN негоциация, което опростява deployment и firewall конфигурацията.

DFA-базираното маршрутизиране, базирано на алгоритъма от „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1], е трети значим резултат. Алгоритъмът постига $O(N)$ времева сложност спрямо дължината на URL-а, независимо от броя на регистрираните маршрути. Експерименталната оценка демонстрира до 100 пъти подобрене спрямо `std::regex` при 500 маршрута.

Типово-безопасното извличане на параметри използва C++20 concepts за compile-time валидация на типовете. Това елиминира цял клас runtime грешки, свързани с некоректно конвертиране на URL параметри.

Платформената независимост е постигната чрез абстрактен I/O слой с оптимизирани имплементации за всяка операционна система: IOCP за Windows, `io_uring` за Linux и `kqueue` за macOS. Всяка имплементация използва native механизмите на платформата за постигане на максимална производителност.

11.1.2 Експериментални резултати

Експерименталната оценка, проведена на система с AMD Ryzen 5 3600 (6 ядра, 12 нишки) и до 1024 конкурентни клиента, демонстрира конкурентна производителност с водещите C++ библиотеки. Coroute постига throughput от 1,378,578 заявки в секунда за прости HTTP отговори на Windows с IOCP backend. Медианната латентност е 212 микросекунди, средната латентност е 743 микросекунди, а минималната латентност достига 59 микросекунди. Error rate-ът е 0%, което потвърждава стабилността на системата при високо натоварване. Консумацията на памет е ефективна — приблизително 23 байта на корутинна за съхранение на състоянието, плюс буфери за I/O операции.

11.1.3 Практическа приложимост

Практическата приложимост на библиотеката е демонстрирана чрез примерното приложение (Flutter Enterprise App) — пълноценна крос-платформена система за управление на задачи. Приложението илюстрира интеграцията на всички ключови компоненти на Coroute: REST API с CRUD операции за задачи и потребители, WebSocket за real-time актуализации, session-based автентикация, сървърно-генерирани изгледи (View) с HTML шаблони, и най-вече — компилирането на целия бекенд като нативна статична/динамична библиотека за изпълнение във Flutter мобилно приложение чрез Dart FFI. Архитектурата на примерното приложение следва утвърдени production patterns и може да служи като референция за изграждане на модерни крос-платформени решения.

11.2 Научен принос

Дипломната работа има следния научен принос, който може да бъде формулиран в три основни направления.

Първият принос е интеграцията на DFA-базирано маршрутизиране в пълноценен уеб библиотека. Алгоритъмът от „Matching Text from Start to Finish Against Multiple Regular Expressions“ [1], първоначално разработен за общо съпоставяне на регулярни изрази, е адаптиран и интегриран в контекста на HTTP маршрутизиране. Експерименталната оценка демонстрира, че теоретичната $O(N)$ сложност се потвърждава в практиката, като времето за съпоставяне остава практически константно при увеличаване на броя на маршрутите от 10 до 500.

Вторият принос е разработването на корутинен изпълнителен модел, специално проектиран за мрежови приложения. Моделът, базиран на типа `Task<T>`, демонстрира как C++20 корутините могат да се използват за изграждане на високопроизводителни асинхронни системи. Ключови иновации включват detached изпълнение за fire-and-forget tasks, continuation chaining за композиране на корутини, и интеграция с платформено-специфични I/O механизми.

Третият принос е демонстрацията на типово-безопасен API дизайн за уеб библиотеки. Използването на C++20 concepts за compile-time валидация на URL параметри е нов подход, който подобрява надеждността на уеб приложенията без runtime overhead.

11.3 Ограничения на изследването

Важно е да се отбележат ограниченията на настоящото изследване. Експерименталната оценка е проведена на localhost, което елиминира мрежовата латентност и може да даде по-оптимистични резултати от реални deployment сценарии. Въпреки че `io_uring` backend-ът за Linux е завършен и тестван, наблюдават се значителни разлики в абсолютната производителност между Windows IOCP и Linux `io_uring`, които се дължат на фундаментални различия в kernel архитектурата. HTTP/3 (QUIC) протоколът не е имплементиран поради значителната му сложност. Тестовите са проведени с ограничен брой конкурентни клиенти (до 1024 при нормални условия, до 10,000 при стрес тестове).

11.4 Насоки за бъдещо развитие

Библиотеката Coroute може да бъде разширена в няколко стратегически направления.

11.4.1 HTTP/3 и QUIC

HTTP/3, базиран на QUIC протокола, представлява следващата еволюция на уеб комуникациите. QUIC решава фундаментален проблем на TCP — Head-of-Line blocking на транспортния слой. При TCP, загубата на един пакет блокира доставката на всички следващи пакети в потока, дори ако те са получени успешно. QUIC, работещ върху UDP, позволява независима доставка на множество потоци, което е критично за ненадеждни мрежови среди.

Интеграцията на QUIC в Coroute би разширила философията на библиотеката за типова безопасност и ефективност към модерните транспортни протоколи. Планираната имплементация ще използва `msquic` или `quiche` като backend, с корутинен API, консистентен с останалата част от библиотеката.

11.4.2 Compile-Time Query Builder

Второ стратегическо направление е интеграцията на compile-time query builder за база данни. Този компонент, разработен в отделен проект, разширява философията на типовата безопасност от URL маршрутизирането към database слоя.

Традиционните ORM библиотеки използват runtime string formatting за генериране на SQL заявки, което въвежда риск от SQL injection и runtime грешки при некоректни заявки. Compile-time query builder-ът използва C++20 `constexpr` и `concepts` за:

- Валидация на SQL синтаксиса по време на компилация
- Типово-безопасно mapping между C++ типове и database колони
- Елиминиране на SQL injection чрез параметризирани заявки
- Zero runtime overhead за генериране на заявки

Интеграцията с Coroute ще предостави асинхронен database клиент с корутинен API:

```
1 auto users = co_await db.query<User>()  
2   .where(User::age > 18)  
3   .order_by(User::name)  
4   .limit(10)  
5   .execute();
```

11.4.3 Допълнителни направления

В дългосрочен план, интересни направления включват: cluster mode с multi-process архитектура и load balancing за хоризонтално мащабиране, gRPC интеграция за microservices сценарии, OpenTelemetry поддръжка за distributed tracing, и публикуване в package managers като vcpkg и Conan за улесняване на инсталацията.

11.5 Заключителни думи

Coroute демонстрира, че модерният C++ (C++20 и по-нови) е подходящ избор за разработка на високопроизводителни уеб приложения. Корутините елиминират традиционните недостатъци на асинхронното програмиране – сложността на callbacks и трудността при debugging – като същевременно запазват ефективността на неблокиращия I/O.

Разработката на Coroute показва, че е възможно да се създаде уеб библиотека, който е едновременно бърз и лесен за използване. Типовата система на C++ може да се използва не само за оптимизация, но и за подобряване на надеждността чрез compile-time проверки. DFA-базираното маршрутизиране демонстрира как алгоритмични иновации могат да подобрят производителността на практически приложения.

Библиотеката е с отворен код и е достъпна за използване и допринасяне от общността. Надяваме се, че Coroute ще послужи като основа за бъдещи проекти, като референтна имплементация за корутинни уеб сървъри, и като принос към развитието на C++ екосистемата за уеб и крос-платформена мобилна разработка.

Не на последно място, концепцията за "Headless Web Engine— изпълнението на пълноценен уеб бекенд директно в процеса на мобилно Flutter приложение — демонстрира изцяло нов подход към изграждането на софтуерни екосистеми. Той свежда до минимум дублирането на бизнес логика, уеднаквява API контрактите между сървъра и клиента, и отваря врати към сигурна, бърза и офлайн-способна архитектура за следващото поколение мобилни и десктоп приложения.

В заключение, настоящата дипломна работа постигна поставените цели и задачи. Създадена е работеща, документирана и тествана библиотека, която може да се използва за изграждане на реални уеб приложения. Резултатите от benchmark тестовете потвърждават, че избраните архитектурни решения водят до висока производителност, сравнима с водещите решения в областта.

Литература

- [1] I. Stankov and A. Tsvetanov, “Matching text from start to finish against multiple regular expressions,” in *32nd National Conference with International Participation “Telecom 2024”*, (Sofia, Bulgaria), pp. 21–22, IEEE, November 2024. 979-8-3503-5500-0/24.
- [2] ISO/IEC, “Programming languages – c++,” International Standard ISO/IEC 14882:2020, International Organization for Standardization, 2020.
- [3] V. Falco, “Boost.beast: Http and websocket built on boost.asio.” <https://www.boost.org/doc/libs/release/libs/beast/>, 2016. Accessed: 2024.
- [4] T. An, “Drogon: A c++14/17/20 based http web application framework.” <https://github.com/drogonframework/drogon>, 2018. Accessed: 2024.
- [5] TechEmpower, “Web framework benchmarks.” <https://www.techempower.com/benchmarks/>, 2024. Accessed: 2024.
- [6] CrowCpp, “Crow: A fast and easy to use microframework for the web.” <https://crowcpp.org/>, 2014. Accessed: 2024.
- [7] S. Leonid, “Oat++: Modern web framework for c++.” <https://oatpp.io/>, 2018. Accessed: 2024.
- [8] Pistache Project, “Pistache: An elegant c++ rest framework.” <http://pistache.io/>, 2015. Accessed: 2024.
- [9] R. Fielding, J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee, “Hypertext transfer protocol – http/1.1,” RFC 2616, IETF, 1999. <https://tools.ietf.org/html/rfc2616>.
- [10] M. Belshe, R. Peon, and M. Thomson, “Hypertext transfer protocol version 2 (http/2),” RFC 7540, IETF, 2015. <https://tools.ietf.org/html/rfc7540>.
- [11] M. Thomson and C. Benfield, “Http/2,” RFC 9113, IETF, 2022. <https://tools.ietf.org/html/rfc9113>.
- [12] R. Peon and H. Ruellan, “Hpack: Header compression for http/2,” RFC 7541, IETF, 2015. <https://tools.ietf.org/html/rfc7541>.
- [13] I. Fette and A. Melnikov, “The websocket protocol,” RFC 6455, IETF, 2011. <https://tools.ietf.org/html/rfc6455>.
- [14] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” RFC 8446, IETF, 2018. <https://tools.ietf.org/html/rfc8446>.
- [15] S. Friedl, A. Popov, A. Langley, and E. Stephan, “Transport layer security (tls)

- application-layer protocol negotiation extension,” RFC 7301, IETF, 2014. <https://tools.ietf.org/html/rfc7301>.
- [16] J. Richter and C. Nasarre, *Windows via C/C++*. Microsoft Press, 5th ed., 2007.
- [17] J. Axboe, “Efficient io with io_uring,” *Linux Kernel Documentation*, 2019. https://kernel.dk/io_uring.pdf.
- [18] FreeBSD Project, “kqueue – kernel event notification mechanism.” FreeBSD Manual Pages, 2000. <https://www.freebsd.org/cgi/man.cgi?kqueue>.
- [19] M. O. Rabin and D. Scott, “Finite automata and their decision problems,” *IBM Journal of Research and Development*, vol. 3, no. 2, pp. 114–125, 1959.
- [20] K. Thompson, “Programming techniques: Regular expression search algorithm,” *Communications of the ACM*, vol. 11, pp. 419–422, June 1968.
- [21] W. Glozer, “wrk - a http benchmarking tool.” <https://github.com/wg/wrk>, 2012. Accessed: 2024.
- [22] P. Nash and M. Horenovsky, “Catch2: A modern, c++-native, test framework for unit-tests.” <https://github.com/catchorg/Catch2>, 2017. Accessed: 2024.
- [23] jagt, “clumsy: An utility for simulating broken network for windows.” <https://jagt.github.io/clumsy/>, 2014. Accessed: 2024. Uses WinDivert library for packet manipulation.